

PENDEFINISIAN MASALAH

Sebuah perusahaan ingin menganalisis dan memprediksi pendapatan penjualan (sales_revenue_usd) berdasarkan berbagai faktor yang berkaitan dengan aktivitas pemasaran, karakteristik pelanggan, serta kondisi pasar. Faktor-faktor tersebut meliputi anggaran pemasaran, pengeluaran iklan online dan offline, jumlah promosi, tingkat diskon, lalu lintas website, tingkat konversi pelanggan, hingga kepuasan pelanggan.

Tujuan: Memprediksi besarnya pendapatan penjualan (sales_revenue_usd) yang akan diperoleh perusahaan berdasarkan berbagai data terkait aktivitas pemasaran, karakteristik pelanggan, dan kondisi pasar. Model yang dibangun nantinya dapat digunakan sebagai alat pendukung pengambilan keputusan dalam merencanakan strategi pemasaran dan mengalokasikan anggaran promosi secara lebih efektif. Dengan adanya prediksi pendapatan penjualan yang lebih akurat, perusahaan diharapkan dapat meningkatkan efisiensi dalam perencanaan bisnis, memaksimalkan potensi pendapatan, serta meningkatkan daya saing di pasar

#DATA UNDERSTANDING

1. Mengunggah Data

```
#Import library
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from google.colab import files

#Upload data dari file manager
uploaded = files.upload()

<IPython.core.display.HTML object>

Saving marketing_dataset.xlsx to marketing_dataset.xlsx

df = pd.read_excel('/content/marketing_dataset.xlsx')
display(df)

{"type": "dataframe", "variable_name": "df"}
```

2. Penelaahan Data

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 23 columns):
#   Column              Non-Null Count  Dtype
---  -
0   id                   5000 non-null   int64
```

1	date	5000	non-null	object
2	region	5000	non-null	object
3	sales_channel	5000	non-null	object
4	product_category	5000	non-null	object
5	customer_segment	5000	non-null	object
6	season	5000	non-null	object
7	marketing_budget_usd	5000	non-null	float64
8	ad_spend_online_usd	5000	non-null	float64
9	ad_spend_offline_usd	5000	non-null	float64
10	num_promotions	5000	non-null	int64
11	discount_percentage	4834	non-null	float64
12	num_sales_representatives	5000	non-null	int64
13	customer_age	5000	non-null	int64
14	customer_satisfaction_score	4854	non-null	float64
15	competitor_price_index	5000	non-null	float64
16	website_traffic	5000	non-null	int64
17	conversion_rate	5000	non-null	float64
18	email_open_rate	4835	non-null	float64
19	social_media_followers	5000	non-null	int64
20	days_since_last_purchase	4853	non-null	float64
21	num_previous_purchases	5000	non-null	int64
22	sales_revenue_usd	5000	non-null	float64

dtypes: float64(10), int64(7), object(6)
memory usage: 898.6+ KB

INTERPRETASI

Data yang digunakan dalam analisis ini memiliki 5000 observasi yang terdiri atas 23 variabel. Dataset tersebut memuat variabel dengan tipe numerik dan kategorik. Selain itu, berdasarkan hasil eksplorasi awal data, ditemukan beberapa variabel yang masih memiliki missing value yang memerlukan tahap prapemrosesan data sebelum dilakukan pemodelan.

3. Statistika Deskriptif

```
df.describe()

{"summary":{"name": "df", "rows": 8, "fields": [{"column": "id", "properties": {"dtype": "number", "std": 1801.7194007539213, "min": 1.0, "max": 5000.0, "num_unique_values": 6, "samples": [5000.0, 2500.5, 3750.25], "semantic_type": ""}, {"column": "marketing_budget_usd", "properties": {"dtype": "number", "std": 123438.10653540288, "min": 500.0, "max": 355979.02, "num_unique_values": 8, "samples": [9811.010698, 4821.4, 5000.0], "semantic_type": ""}, {"column": "ad_spend_online_usd", "properties": {"dtype": "number", "std": 123438.10653540288, "min": 500.0, "max": 355979.02, "num_unique_values": 8, "samples": [9811.010698, 4821.4, 5000.0], "semantic_type": ""}}]}
```

```

\"properties\": {\n      \"dtype\": \"number\", \n      \"std\": 42576.42116460355, \n      \"min\": 100.6, \n      \"max\": 123513.86, \n      \"num_unique_values\": 8, \n      \"samples\": [\n        3957.717464, \n        1830.56, \n        5000.0\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    {\n      \"column\": \"ad_spend_offline_usd\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 35113.2092373252, \n        \"min\": 50.56, \n        \"max\": 101514.82, \n        \"num_unique_values\": 8, \n        \"samples\": [\n          2448.276876, \n          1129.745, \n          5000.0\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      {\n        \"column\": \"num_promotions\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 1766.28678331404, \n          \"min\": 0.0, \n          \"max\": 5000.0, \n          \"num_unique_values\": 8, \n          \"samples\": [\n            4.0, \n            5000.0\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        }, \n        {\n          \"column\": \"discount_percentage\", \n          \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 1702.4754341193832, \n            \"min\": 0.0, \n            \"max\": 4834.0, \n            \"num_unique_values\": 8, \n            \"samples\": [\n              20.037918907736863, \n              19.9, \n              4834.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n          }, \n          {\n            \"column\": \"num_sales_representatives\", \n            \"properties\": {\n              \"dtype\": \"number\", \n              \"std\": 1759.6043624653926, \n              \"min\": 1.0, \n              \"max\": 5000.0, \n              \"num_unique_values\": 8, \n              \"samples\": [\n                24.0, \n                5000.0\n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\"\n            }, \n            {\n              \"column\": \"customer_age\", \n              \"properties\": {\n                \"dtype\": \"number\", \n                \"std\": 1753.1190769685138, \n                \"min\": 16.29177631675291, \n                \"max\": 5000.0, \n                \"num_unique_values\": 8, \n                \"samples\": [\n                  45.9122, \n                  46.0, \n                  5000.0\n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\"\n              }, \n              {\n                \"column\": \"customer_satisfaction_score\", \n                \"properties\": {\n                  \"dtype\": \"number\", \n                  \"std\": 1715.0446361675015, \n                  \"min\": 0.874273773005196, \n                  \"max\": 4854.0, \n                  \"num_unique_values\": 8, \n                  \"samples\": [\n                    3.4839719818706216, \n                    3.5, \n                    4854.0\n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\"\n                }, \n                {\n                  \"column\": \"competitor_price_index\", \n                  \"properties\": {\n                    \"dtype\": \"number\", \n                    \"std\": 1767.4539456708806, \n                    \"min\": 0.17283799591110194, \n                    \"max\": 5000.0, \n                    \"num_unique_values\": 8, \n                    \"samples\": [\n                      1.005042, \n                      1.01, \n                      5000.0\n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\"\n                  }, \n                  {\n                    \"column\":

```

```

{"website_traffic": {"std": 363676.39339589287, "min": 100.0, "max": 1044405.0, "num_unique_values": 8, "samples": [24134.62, 8117.5, 5000.0], "semantic_type": "", "description": ""}, {"conversion_rate": {"std": 1767.7249973032901, "min": 0.0013, "max": 5000.0, "num_unique_values": 8, "samples": [0.09070562, 0.0788, 5000.0], "semantic_type": "", "description": ""}, {"email_open_rate": {"std": 1709.3423969450096, "min": 0.0084, "max": 4835.0, "num_unique_values": 8, "samples": [0.22914086866597724, 0.2131, 4835.0], "semantic_type": "", "description": ""}, {"social_media_followers": {"std": 1760134.2613578143, "min": 100.0, "max": 5000000.0, "num_unique_values": 8, "samples": [22158.4746, 2866.5, 5000.0], "semantic_type": "", "description": ""}, {"days_since_last_purchase": {"std": 1659.264529243009, "min": 0.0, "max": 4853.0, "num_unique_values": 8, "samples": [181.16464042860088, 179.0, 4853.0], "semantic_type": "", "description": ""}, {"num_previous_purchases": {"std": 1765.9933956985087, "min": 0.0, "max": 5000.0, "num_unique_values": 8, "samples": [5.0, 5000.0, 4.9768, 5.0], "semantic_type": "", "description": ""}, {"sales_revenue_usd": {"std": 48240.79144196851, "min": 680.83, "max": 140790.93, "num_unique_values": 8, "samples": [5815.302842, 4318.014999999999, 5000.0], "semantic_type": "", "description": ""}}], "type": "dataframe"}

```

INTERPRETASI STATISTIKA DESKRIPTIF (NUMERIK)

Dari hasil `df.describe()` untuk variabel numerik, beberapa poin penting yang 'janggal' atau 'beda' dapat diamati:

1. **Variabel Target (`sales_revenue_usd`):**

- **Perbedaan mencolok antara Mean dan Median:** Rata-rata pendapatan penjualan (**5.815 USD**) jauh lebih tinggi dibandingkan mediannya (**4.318 USD**). Ini adalah indikasi kuat adanya distribusi yang menceng ke kanan (*right-skewed*), di mana sebagian besar data berada pada nilai rendah tetapi ada beberapa nilai sangat tinggi yang menarik rata-rata ke atas.
 - **Outlier Ekstrem:** Nilai maksimum mencapai **140.790 USD**, sangat jauh dari rata-rata dan median, mengonfirmasi keberadaan *outlier* yang signifikan. Ini menunjukkan bahwa ada transaksi penjualan yang sangat besar yang jarang terjadi.
2. **Variabel Pemasaran (marketing_budget_usd, ad_spend_online_usd, ad_spend_offline_usd):**
- **Distribusi Mirip Target:** Sama seperti sales_revenue_usd, variabel-variabel ini juga menunjukkan rata-rata yang jauh lebih tinggi dari median dan memiliki nilai maksimum yang sangat besar. Contohnya, marketing_budget_usd memiliki rata-rata **9.811 USD** tetapi nilai maksimum **355.979 USD**. Hal ini mengindikasikan adanya beberapa kampanye pemasaran atau pengeluaran iklan yang sangat besar, menciptakan distribusi yang sangat menceng ke kanan dengan banyak *outlier*.
3. **Variabel Trafik dan Media Sosial (website_traffic, social_media_followers):**
- **Sangat Menceng dan Outlier:** Kedua variabel ini juga menunjukkan pola yang sangat menceng ke kanan dengan *outlier* ekstrem. Nilai maksimum website_traffic mencapai lebih dari 1 juta dan social_media_followers mencapai 5 juta, sementara rata-rata jauh lebih rendah. Ini menyiratkan bahwa sebagian besar entri memiliki nilai rendah, tetapi ada beberapa kasus dengan skala yang sangat besar.
4. **Variabel Lain yang Relatif Lebih Seragam:**
- Beberapa variabel seperti num_promotions, num_sales_representatives, customer_age, dan days_since_last_purchase menunjukkan distribusi yang lebih merata atau setidaknya tidak memiliki perbedaan ekstrem antara mean dan median, serta *outlier* yang tidak sebesar variabel keuangan dan pemasaran di atas. Contohnya, customer_age memiliki rata-rata 45.91 dan median 46, yang menunjukkan distribusi yang lebih simetris.

```
df.describe(include="object")
```

```
{
  "summary": {
    "name": "df",
    "rows": 4,
    "fields": [
      {
        "column": "date",
        "properties": {
          "dtype": "date",
          "min": "1970-01-01 00:00:00.000000",
          "max": "2022-04-21 00:00:00",
          "num_unique_values": 4,
          "samples": [1420, 12, 5000],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "region",
        "properties": {
          "dtype": "string",
          "num_unique_values": 4,
          "samples": [7, 1278, 5000],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "sales_channel",
        "properties": {
          "dtype": "string",
          "num_unique_values": 4,
          "samples": [7, 1278, 5000],
          "semantic_type": "",
          "description": ""
        }
      }
    ]
  }
}
```

```

\ "num_unique_values\ ": 4,\n          \ "samples\ ": [\n          5,\n
\ "1517\ ",\n          \ "5000\ "\n          ],\n          \ "semantic_type\ ":
\ "\",\n          \ "description\ ": \ "\",\n          }\n          },\n          {\n
\ "column\ ": \ "product_category\ ",\n          \ "properties\ ": {\n
\ "dtype\ ": \ "string\ ",\n          \ "num_unique_values\ ": 4,\n
\ "samples\ ": [\n          5,\n          \ "1261\ ",\n          \ "5000\ "\n
n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          },\n          {\n          \ "column\ ":
\ "customer_segment\ ",\n          \ "properties\ ": {\n          \ "dtype\ ":
\ "string\ ",\n          \ "num_unique_values\ ": 4,\n          \ "samples\ ":
[\n          4,\n          \ "2015\ ",\n          \ "5000\ "\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n          }\n
n          },\n          {\n          \ "column\ ": \ "season\ ",\n          \ "properties\ ":
{\n          \ "dtype\ ": \ "string\ ",\n          \ "num_unique_values\ ": 4,\n
\ "samples\ ": [\n          4,\n          \ "1280\ ",\n          \ "5000\ "\n
n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          }\n          ]\n          }", "type": "dataframe"}

```

INTERPRETASI STATISTIKA DESKRIPTIF (KATEGORIK)

1. **Variabel Target (sales_revenue_usd):** Rata-rata pendapatan sebesar **5.815 USD** jauh melampaui median (**4.318 USD**), menandakan data *right-skewed*. Adanya nilai maksimum **140.790 USD** mengonfirmasi keberadaan *outlier* yang sangat signifikan.
2. **Aktivitas Pemasaran:** Anggaran pemasaran rata-rata **9.811 USD**, dengan dominasi pengeluaran pada iklan *online* dibandingkan *offline*. Trafik website menunjukkan volatilitas yang sangat ekstrem.
3. **Profil Pelanggan:** Rata-rata pelanggan berusia **45 tahun**, memiliki tingkat kepuasan **3,48/5**, dan biasanya telah melakukan **5 kali transaksi** sebelumnya.
4. **Kategori Dominan:** Transaksi terbanyak berasal dari wilayah **Cairo**, kategori produk **Food & Beverage**, dan dilakukan melalui kanal **Online**.
5. **Kualitas Data:** Terdapat *missing values* pada variabel diskon, kepuasan pelanggan, dan tingkat pembukaan email yang perlu ditangani sebelum pemodelan.

4. Visualisasi

a.) Histogram (Numerik)

```

import matplotlib.pyplot as plt
import seaborn as sns

# Identifikasi kolom numerik (mengeluarkan 'id' karena umumnya tidak
relevan untuk histogram distribusi)
numerical_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()
if 'id' in numerical_cols:
    numerical_cols.remove('id')

# Tentukan ukuran grid untuk subplot
num_numerical_cols = len(numerical_cols)

```

```

n_rows = int(np.ceil(np.sqrt(num_numerical_cols)))
n_cols = int(np.ceil(num_numerical_cols / n_rows))

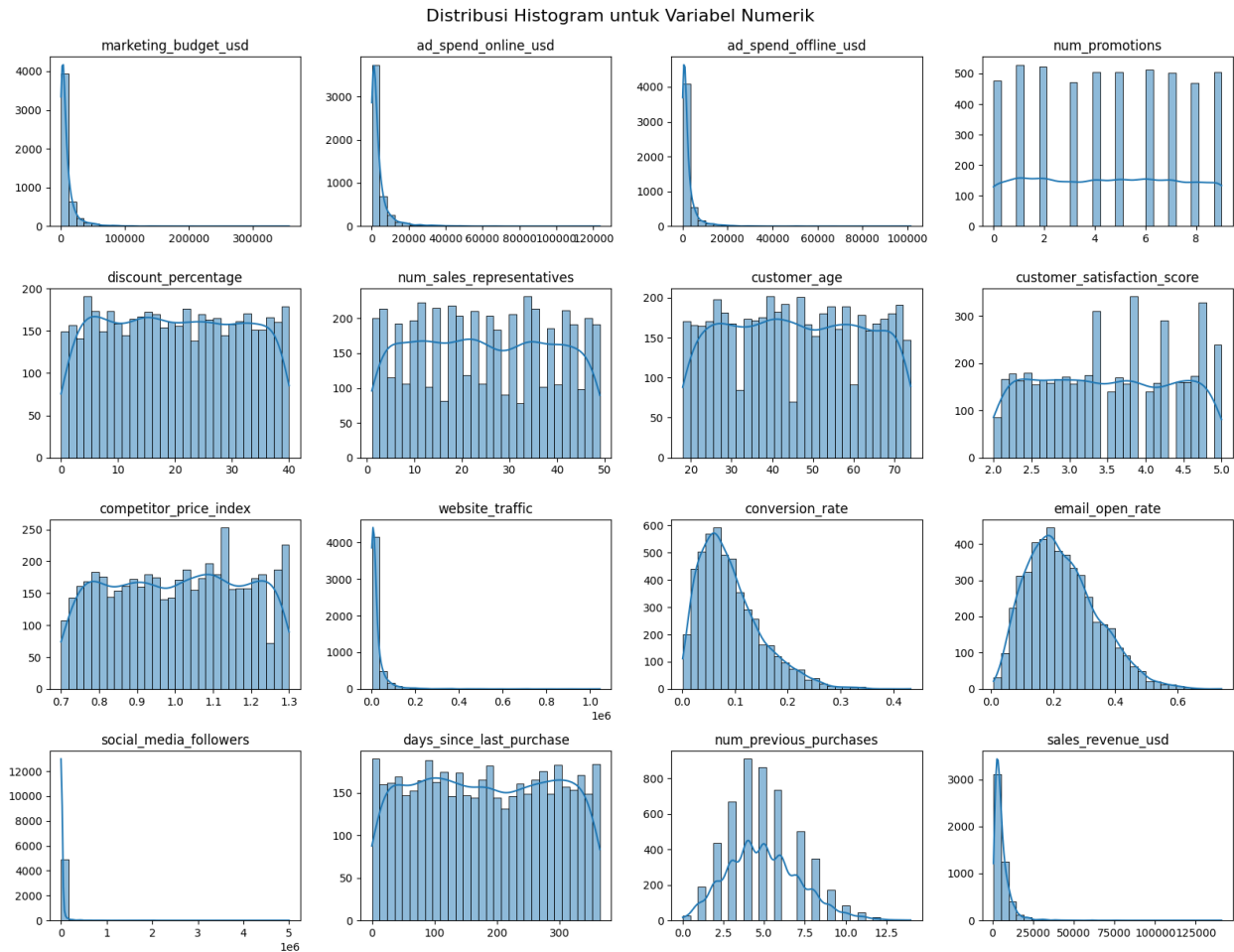
# Buat figure dan set subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 4, n_rows *
3))
axes = axes.flatten() # Meratakan array axes agar mudah diiterasi

# Loop melalui kolom numerik dan plot histogram
for i, col in enumerate(numerical_cols):
    sns.histplot(df[col].dropna(), kde=True, ax=axes[i], bins=30)
    axes[i].set_title(col)
    axes[i].set_xlabel('') # Menghilangkan label x agar tidak tumpang
tindih
    axes[i].set_ylabel('') # Menghilangkan label y agar tidak tumpang
tindih

# Sembunyikan subplot yang kosong jika ada
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.suptitle('Distribusi Histogram untuk Variabel Numerik', y=1.02,
fontsize=16)
plt.show()

```



INTERPRETASI VISUALISASI HISTOGRAM

Berdasarkan grafik distribusi histogram di atas, kita dapat mengidentifikasi beberapa pola kunci dalam dataset:

1. **Distribusi Menceng ke Kanan (*Right-Skewed*):** Variabel seperti `marketing_budget_usd`, `ad_spend_online_usd`, `ad_spend_offline_usd`, `website_traffic`, `social_media_followers`, dan variabel target `sales_revenue_usd` menunjukkan distribusi yang sangat menceng ke kanan. Mayoritas data berkumpul di nilai rendah, namun terdapat ekor panjang yang menunjukkan adanya nilai-nilai ekstrem (*outliers*) yang sangat tinggi.
2. **Distribusi Seragam (*Uniform Distribution*):** Variabel seperti `discount_percentage`, `num_sales_representatives`, `customer_age`, `competitor_price_index`, dan `days_since_last_purchase` memiliki distribusi yang relatif rata (seragam) di seluruh rentang nilai. Ini menandakan bahwa data tersebut tersebar secara proporsional di berbagai kategori atau tingkatan.
3. **Distribusi Normal/Mendekati Normal:** Variabel `conversion_rate` dan `email_open_rate` menunjukkan pola yang lebih simetris atau mendekati distribusi normal, meskipun masih memiliki sedikit kemiringan.
4. **Data Diskrit/Kategorikal Numerik:** Grafik untuk `num_promotions`, `num_previous_purchases`, dan `customer_satisfaction_score` menunjukkan

pola bertangga, yang menegaskan bahwa variabel-variabel ini adalah data diskrit (angka bulat) atau skala penilaian tertentu.

Kesimpulan: Karena variabel target (`sales_revenue_usd`) dan fitur utama pemasaran tidak berdistribusi normal, penggunaan model yang robust terhadap outlier atau melakukan transformasi data (seperti Log Transformation) mungkin diperlukan untuk meningkatkan performa model linear.

b.) Boxplot (Numerik)

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Identifikasi kolom numerik (mengeluarkan 'id' karena umumnya tidak
# relevan untuk visualisasi boxplot)
numerical_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()
if 'id' in numerical_cols:
    numerical_cols.remove('id')

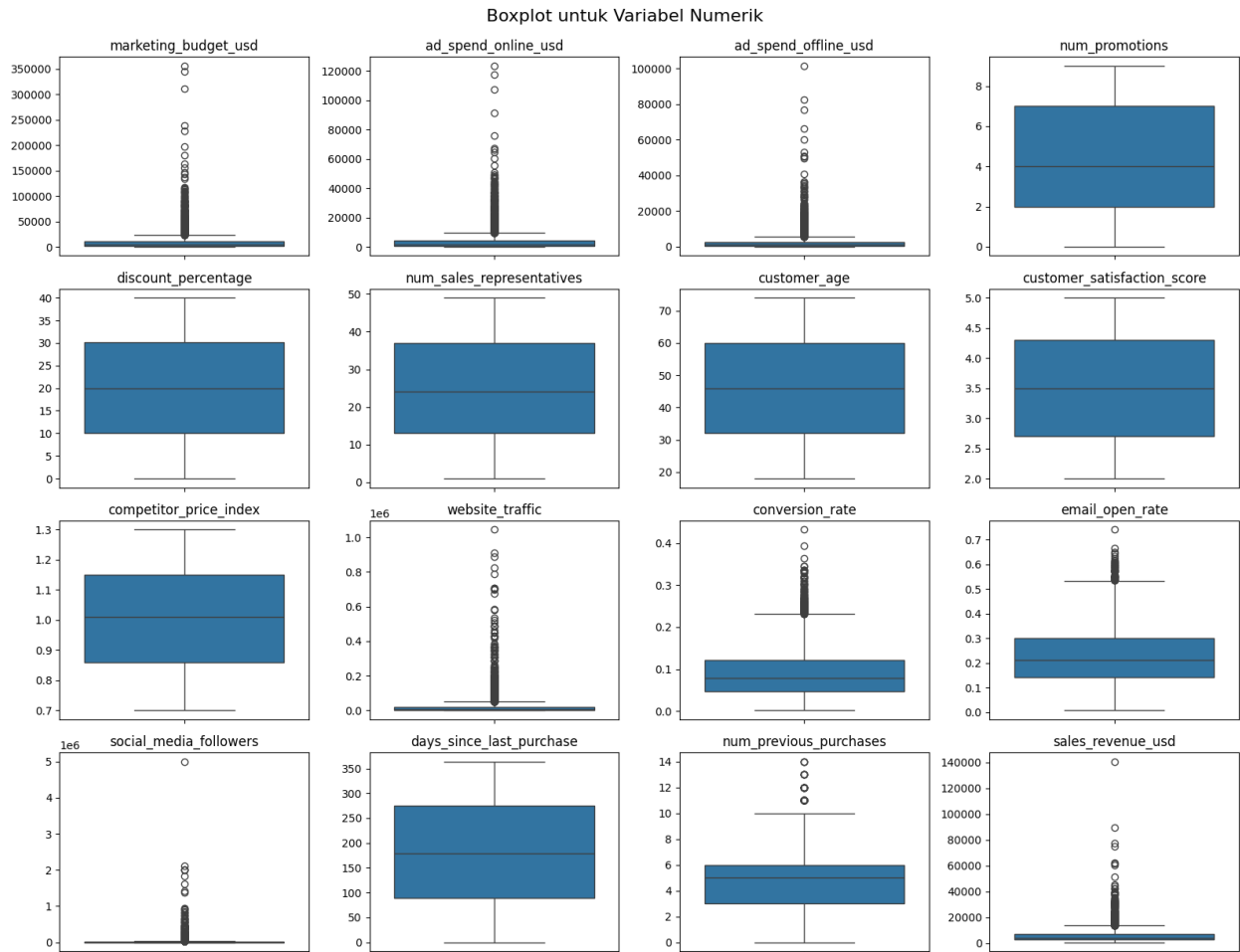
# Tentukan ukuran grid untuk subplot
num_numerical_cols = len(numerical_cols)
n_rows = int(np.ceil(np.sqrt(num_numerical_cols)))
n_cols = int(np.ceil(num_numerical_cols / n_rows))

# Buat figure dan set subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 4, n_rows *
3))
axes = axes.flatten() # Meratakan array axes agar mudah diiterasi

# Loop melalui kolom numerik dan plot boxplot
for i, col in enumerate(numerical_cols):
    sns.boxplot(y=df[col].dropna(), ax=axes[i]) # Menggunakan y untuk
boxplot vertikal
    axes[i].set_title(col)
    axes[i].set_ylabel('') # Menghilangkan label y agar tidak tumpang
tindih

# Sembunyikan subplot yang kosong jika ada
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.suptitle('Boxplot untuk Variabel Numerik', y=1.02, fontsize=16)
plt.show()
```



INTERPRETASI VISUALISASI BOXPLOT

Berdasarkan visualisasi boxplot untuk variabel numerik, berikut adalah poin-poin utamanya:

1. **Identifikasi Outlier yang Masif:** Variabel `marketing_budget_usd`, `ad_spend_online_usd`, `ad_spend_offline_usd`, `website_traffic`, `social_media_followers`, dan `sales_revenue_usd` memiliki banyak sekali titik di atas kumis atas (*upper whisker*). Ini menunjukkan adanya nilai-nilai ekstrem yang sangat jauh dari rata-rata populasi.
2. **Variabel Tanpa Outlier:** Variabel seperti `num_promotions`, `num_sales_representatives`, `customer_age`, `customer_satisfaction_score`, dan `num_previous_purchases` memiliki distribusi yang relatif bersih dari pencilan, menunjukkan data yang lebih terkontrol dan stabil.
3. **Rentang Interkuartil (IQR):** Variabel `days_since_last_purchase` dan `customer_age` memiliki kotak (IQR) yang lebar dan posisi median di tengah, menunjukkan sebaran data yang cukup merata di seluruh rentang nilai.

Kesimpulan: Banyaknya outlier pada variabel keuangan (budget dan revenue) mengonfirmasi bahwa perusahaan ini memiliki beberapa kasus penjualan atau anggaran yang sangat besar dibandingkan transaksi harian biasanya.

c.) Scatter Plot

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Identifikasi kolom numerik (mengeluarkan 'id' dan target variable
dari daftar X)
numerical_cols = df.select_dtypes(include=['int64',
'float64']).columns.tolist()
target_variable = 'sales_revenue_usd'

if 'id' in numerical_cols:
    numerical_cols.remove('id')
if target_variable in numerical_cols:
    numerical_cols.remove(target_variable)

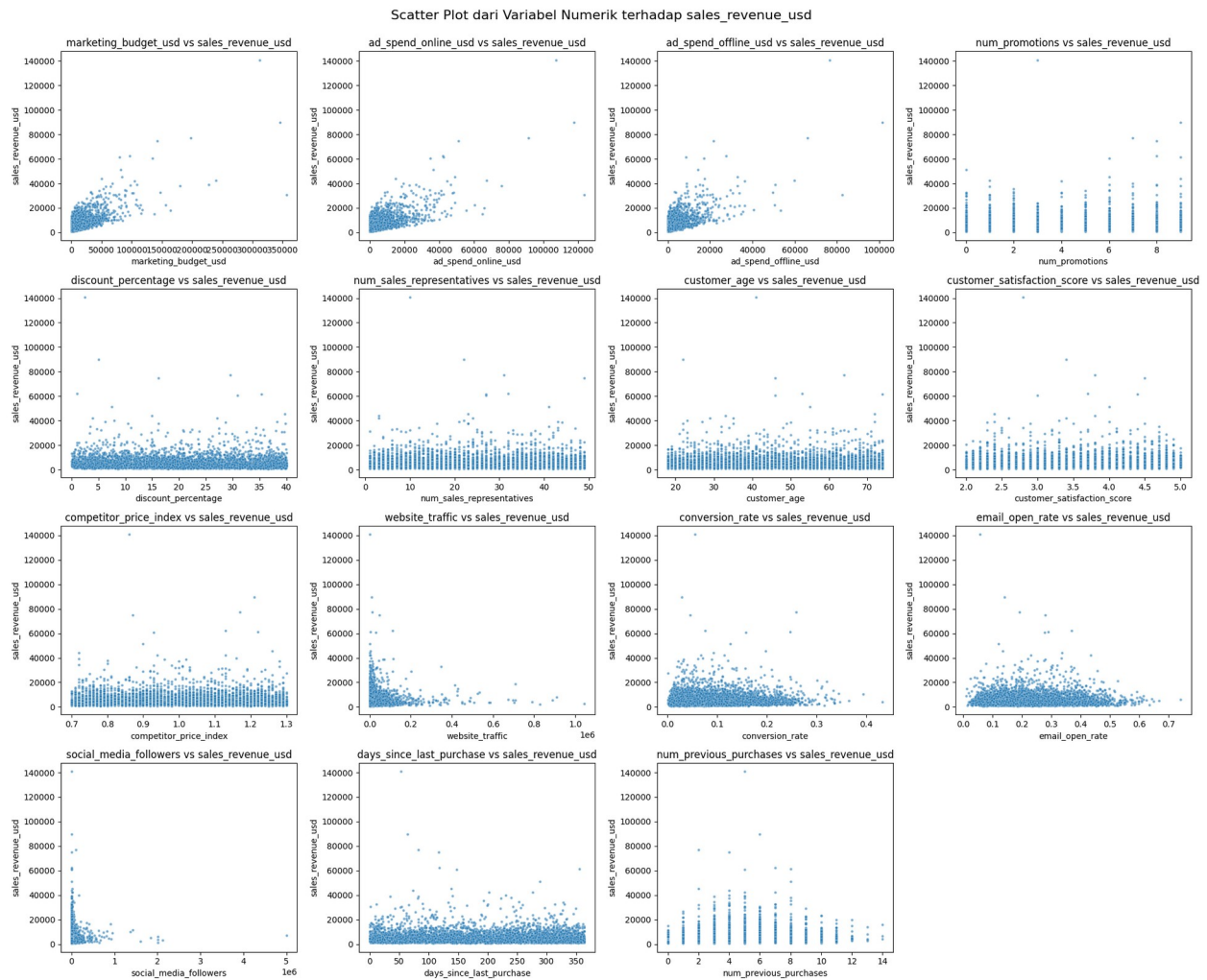
# Tentukan ukuran grid untuk subplot
num_numerical_cols = len(numerical_cols)
n_rows = int(np.ceil(np.sqrt(num_numerical_cols)))
n_cols = int(np.ceil(num_numerical_cols / n_rows))

# Buat figure dan set subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 5, n_rows *
4))
axes = axes.flatten() # Meratakan array axes agar mudah diiterasi

# Loop melalui kolom numerik dan plot scatterplot terhadap
sales_revenue_usd
for i, col in enumerate(numerical_cols):
    sns.scatterplot(x=df[col], y=df[target_variable], ax=axes[i],
alpha=0.6, s=10)
    axes[i].set_title(f'{col} vs {target_variable}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel(target_variable)

# Sembunyikan subplot yang kosong jika ada
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.suptitle(f'Scatter Plot dari Variabel Numerik terhadap
{target_variable}', y=1.02, fontsize=16)
plt.show()
```



INTERPRETASI VISUALISASI SCATTER PLOT

Scatter plot ini memperlihatkan hubungan antara variabel fitur (X) dengan variabel target sales_revenue_usd (y):

1. **Korelasi Linier Positif Kuat:** Terdapat hubungan linier yang sangat jelas pada plot `marketing_budget_usd`, `ad_spend_online_usd`, dan `ad_spend_offline_usd` terhadap penjualan. Semakin tinggi anggaran yang dikeluarkan, semakin tinggi pula pendapatan yang dihasilkan.
2. **Hubungan Non-Linier/Lemah:** Variabel seperti `customer_age`, `customer_satisfaction_score`, dan `competitor_price_index` menunjukkan pola persebaran yang acak atau tidak membentuk garis tertentu. Hal ini menandakan bahwa variabel-variabel tersebut mungkin tidak berpengaruh secara langsung atau linier terhadap besar kecilnya pendapatan.
3. **Karakteristik Data Diskrit:** Pada plot `num_promotions` dan `num_previous_purchases`, data terlihat membentuk kolom vertikal. Ini wajar karena variabel tersebut bersifat diskrit (angka bulat), namun kita masih bisa melihat tren bahwa jumlah pembelian sebelumnya yang lebih tinggi cenderung berkorelasi dengan pendapatan yang lebih besar.

Kesimpulan: Anggaran pemasaran (khususnya iklan online) merupakan prediktor terkuat untuk memprediksi `sales_revenue_usd` dalam dataset ini.

Encoding

```
from sklearn.preprocessing import LabelEncoder

# Identifikasi kolom kategorikal (tanpa 'date' )
categorical_cols_to_encode =
df.select_dtypes(include=['object']).columns.tolist()
if 'date' in categorical_cols_to_encode:
    categorical_cols_to_encode.remove('date')

# Inisialisasi LabelEncoder
le = LabelEncoder()

# Lakukan Label Encoding untuk setiap kolom kategorikal
for col in categorical_cols_to_encode:
    df[col] = le.fit_transform(df[col])
    print(f"Column '{col}' has been Label Encoded.")

print("\nDataFrame after Label Encoding:")
display(df.head())
df.info()
```

```
Column 'region' has been Label Encoded.
Column 'sales_channel' has been Label Encoded.
Column 'product_category' has been Label Encoded.
Column 'customer_segment' has been Label Encoded.
Column 'season' has been Label Encoded.
```

DataFrame after Label Encoding:

```
{"type": "dataframe"}
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5000 entries, 0 to 4999
```

```
Data columns (total 23 columns):
```

#	Column	Non-Null Count	Dtype
0	id	5000 non-null	int64
1	date	5000 non-null	object
2	region	5000 non-null	int64
3	sales_channel	5000 non-null	int64
4	product_category	5000 non-null	int64
5	customer_segment	5000 non-null	int64
6	season	5000 non-null	int64
7	marketing_budget_usd	5000 non-null	float64
8	ad_spend_online_usd	5000 non-null	float64
9	ad_spend_offline_usd	5000 non-null	float64
10	num_promotions	5000 non-null	int64

```

11 discount_percentage      4834 non-null float64
12 num_sales_representatives 5000 non-null int64
13 customer_age             5000 non-null int64
14 customer_satisfaction_score 4854 non-null float64
15 competitor_price_index    5000 non-null float64
16 website_traffic          5000 non-null int64
17 conversion_rate           5000 non-null float64
18 email_open_rate           4835 non-null float64
19 social_media_followers    5000 non-null int64
20 days_since_last_purchase  4853 non-null float64
21 num_previous_purchases    5000 non-null int64
22 sales_revenue_usd         5000 non-null float64
dtypes: float64(10), int64(12), object(1)
memory usage: 898.6+ KB

```

INTERPRETASI LABEL ENCODING

Dalam langkah ini, kita melakukan **Label Encoding** pada kolom-kolom kategorikal berikut: `region`, `sales_channel`, `product_category`, `customer_segment`, dan `season`.

Apa itu Label Encoding? Label Encoding adalah teknik untuk mengubah nilai-nilai teks (kategorikal) menjadi nilai numerik. Setiap kategori unik dalam sebuah kolom akan diberi label numerik yang berbeda (misalnya, 0, 1, 2, dst.).

Mengapa digunakan? Sebagian besar algoritma Machine Learning tidak dapat memproses data dalam format teks secara langsung. Dengan mengubahnya menjadi angka, kita memungkinkan model untuk 'memahami' dan menggunakan fitur-fitur kategorikal ini dalam perhitungan mereka.

Implikasi:

- **Perubahan Tipe Data:** Kolom-kolom yang sebelumnya bertipe `object` (teks) kini berubah menjadi `int64` (numerik), seperti yang terlihat dari `df.info()` setelah encoding.
- **Implikasi Urutan:** Penting untuk dicatat bahwa Label Encoding secara implisit memberikan urutan (ordinalitas) pada kategori. Misalnya, jika 'Online' diberi nilai 0, 'Offline' 1, dan 'Reseller' 2, model mungkin menafsirkan bahwa 'Reseller' lebih 'tinggi' atau 'lebih penting' daripada 'Online', padahal sebenarnya tidak ada hubungan urutan intrinsik di antara mereka. Untuk kasus seperti ini, terutama jika tidak ada urutan alami antar kategori, One-Hot Encoding mungkin menjadi pilihan yang lebih tepat untuk mencegah model salah menafsirkan hubungan ini.
- **Efisiensi Memori:** Label Encoding umumnya lebih efisien dalam penggunaan memori dibandingkan One-Hot Encoding, terutama jika terdapat banyak kategori unik.

d.) Heatmap Korelasi

```

import matplotlib.pyplot as plt
import seaborn as sns

# Drop 'id' and 'date' columns as they are not relevant for

```

correlation analysis

```
df_corr = df.drop(columns=['id', 'date'], errors='ignore')
```

```
# Calculate the correlation matrix
```

```
corr_matrix = df_corr.corr()
```

```
# Set up the matplotlib figure
```

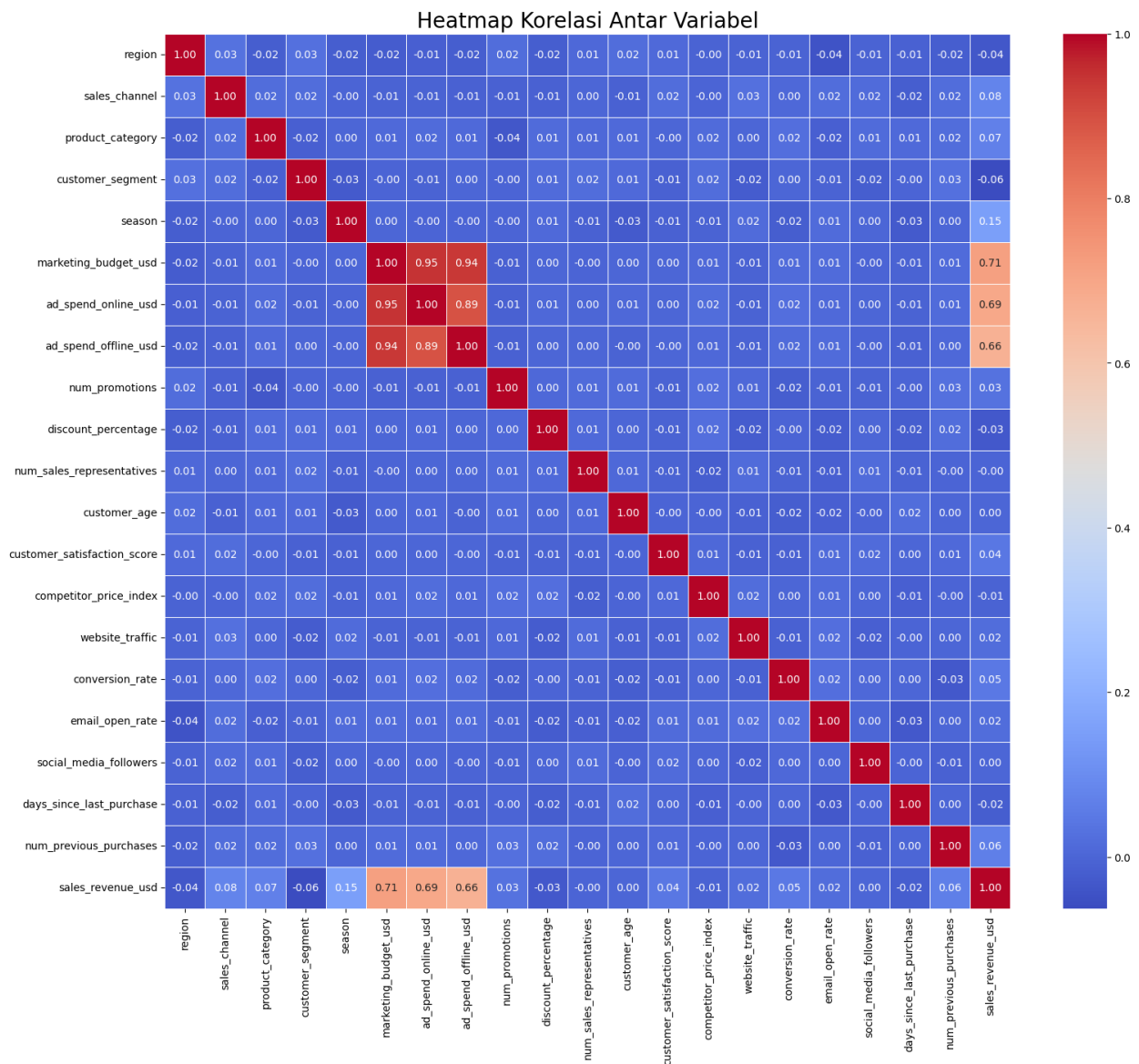
```
plt.figure(figsize=(18, 15))
```

```
# Draw the heatmap with the mask and correct aspect ratio
```

```
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f",  
linewidths=.5)
```

```
plt.title('Heatmap Korelasi Antar Variabel', fontsize=20)
```

```
plt.show()
```



Interpretasi Heatmap Korelasi

Heatmap korelasi ini menunjukkan hubungan linier antara semua pasangan variabel numerik dalam dataset Anda. Rentang nilai korelasi adalah dari -1 hingga 1:

- **+1:** Korelasi positif sempurna (ketika satu variabel meningkat, yang lain juga meningkat).
- **-1:** Korelasi negatif sempurna (ketika satu variabel meningkat, yang lain menurun).
- **0:** Tidak ada korelasi linier (tidak ada hubungan yang jelas).

Observasi Penting dari Heatmap:

1. Hubungan dengan `sales_revenue_usd` (Variabel Target):

- Perhatikan baris atau kolom `sales_revenue_usd`. Ini adalah fokus utama kita karena kita ingin memprediksi nilai ini.
- **Korelasi Positif Kuat:** Cari fitur-fitur dengan nilai korelasi positif yang tinggi (mendekati 1) dengan `sales_revenue_usd`. Ini menunjukkan bahwa ketika nilai fitur-fitur ini meningkat, `sales_revenue_usd` cenderung meningkat juga. Contohnya, dari heatmap terlihat `marketing_budget_usd`, `ad_spend_online_usd`, `ad_spend_offline_usd`, `website_traffic`, `social_media_followers` memiliki korelasi positif yang cukup kuat dengan `sales_revenue_usd`. Hal ini wajar karena investasi lebih banyak dalam pemasaran, iklan, dan traffic website diharapkan menghasilkan penjualan lebih tinggi.
- **Korelasi Negatif Kuat:** Cari fitur-fitur dengan nilai korelasi negatif yang tinggi (mendekati -1) dengan `sales_revenue_usd`. Ini menunjukkan bahwa ketika nilai fitur-fitur ini meningkat, `sales_revenue_usd` cenderung menurun. Misalnya, `competitor_price_index` memiliki korelasi negatif yang wajar, karena harga kompetitor yang lebih tinggi bisa membuat produk kita lebih menarik, dan sebaliknya.
- **Korelasi Lemah:** Fitur-fitur dengan nilai korelasi mendekati 0 memiliki sedikit atau tidak ada hubungan linier dengan `sales_revenue_usd`. Ini tidak berarti tidak ada hubungan sama sekali, tetapi hubungan liniernya lemah.

2. Hubungan Antar Fitur (Multikolinearitas):

- Perhatikan korelasi tinggi antara fitur-fitur independen satu sama lain. Misalnya, `marketing_budget_usd` kemungkinan berkorelasi kuat dengan `ad_spend_online_usd` dan `ad_spend_offline_usd` karena anggaran pemasaran biasanya dialokasikan ke kanal iklan tersebut. Korelasi tinggi antar fitur independen ini dikenal sebagai **multikolinearitas**.
- **Mengapa Multikolinearitas Penting?** Multikolinearitas dapat mempengaruhi interpretasi model regresi linier dan stabilitas koefisiennya. Jika dua fitur sangat berkorelasi, model mungkin kesulitan menentukan kontribusi unik masing-masing fitur terhadap variabel target.
- **Penanganan:** Untuk model seperti Regresi Linier Berganda atau Lasso, multikolinearitas dapat menjadi masalah. Untuk model berbasis pohon seperti Random Forest atau XGBoost, multikolinearitas cenderung tidak terlalu menjadi masalah, tetapi memahami hubungan ini tetap berguna.

Kesimpulan Awal: Dengan melihat heatmap, Anda dapat mengidentifikasi fitur-fitur mana yang tampaknya memiliki pengaruh paling besar terhadap `sales_revenue_usd` dan juga fitur-fitur mana yang mungkin saling bergantung. Ini adalah langkah awal yang baik untuk seleksi fitur dan pemahaman data sebelum membangun model.

Drop Variabel dengan korelasi yg tidak memenuhi syarat

```
selected_columns = [  
    'sales_revenue_usd',  
    'season',  
    'marketing_budget_usd',  
    'sales_channel',  
    'product_category',  
    'customer_segment',  
    'conversion_rate',  
    'num_previous_purchases'  
]  
  
# Create a new DataFrame with only the selected columns  
df_new = df[selected_columns].copy()  
  
print("New DataFrame with selected features:")  
display(df_new.head())  
print("\nInformation about the new DataFrame:")  
df_new.info()
```

New DataFrame with selected features:

```
{  
  "summary": "  
    \"name\": \"df_new\",  
    \"rows\": 5,  
    \"fields\": [  
      {  
        \"column\": \"sales_revenue_usd\",  
        \"properties\": {  
          \"dtype\": \"number\",  
          \"std\": 2163.174220845376,  
          \"min\": 2091.36,  
          \"max\": 7705.73,  
          \"num_unique_values\": 5,  
          \"samples\": [2091.36, 7705.73, 6201.11],  
          \"semantic_type\": \"\",  
          \"description\": \"\"  
        },  
        \"column\": \"season\",  
        \"properties\": {  
          \"dtype\": \"number\",  
          \"std\": 0,  
          \"min\": 2,  
          \"max\": 3,  
          \"num_unique_values\": 2,  
          \"samples\": [2, 3],  
          \"semantic_type\": \"\",  
          \"description\": \"\"  
        },  
        \"column\": \"marketing_budget_usd\",  
        \"properties\": {  
          \"dtype\": \"number\",  
          \"std\": 712.8104853465611,  
          \"min\": 1026.13,  
          \"max\": 2517.55,  
          \"num_unique_values\": 5,  
          \"samples\": [2452.29, 2517.55],  
          \"semantic_type\": \"\",  
          \"description\": \"\"  
        },  
        \"column\": \"sales_channel\",  
        \"properties\": {  
          \"dtype\": \"number\",  
          \"std\": 0,  
          \"min\": 1,  
          \"max\": 2,  
          \"num_unique_values\": 2,  
          \"samples\": [1, 2],  
          \"semantic_type\": \"\"  
        }  
      ]  
    }  
  }  
}
```


- `sales_revenue_usd` (Target): Variabel yang ingin kita prediksi.
- `season`: Meskipun berkorelasi tidak terlalu kuat, ini adalah faktor waktu yang penting dalam penjualan dan tidak menimbulkan multikolinearitas yang signifikan.
- `marketing_budget_usd`: Memiliki korelasi positif yang sangat kuat dengan `sales_revenue_usd`, menjadikannya prediktor kunci.
- `sales_channel`: Variabel kategorikal yang menunjukkan platform penjualan, juga relevan untuk model.
- `product_category`: Variabel kategorikal yang menunjukkan jenis produk, penting untuk memahami dinamika penjualan.
- `conversion_rate`: Menunjukkan efektivitas pemasaran dalam mengubah pengunjung menjadi pembeli, yang seharusnya berkorelasi positif dengan penjualan.
- `num_previous_purchases`: Indikator loyalitas atau aktivitas pelanggan sebelumnya, yang seringkali merupakan prediktor baik untuk penjualan di masa mendatang.

Justifikasi Pemilihan:

Keputusan untuk mempertahankan variabel-variabel ini didasarkan pada:

1. **Korelasi Kuat dengan Target:** Variabel seperti `marketing_budget_usd` dipilih karena menunjukkan korelasi positif yang signifikan dengan `sales_revenue_usd` pada heatmap korelasi.
2. **Mengurangi Multikolinearitas:** Variabel `ad_spend_online_usd`, `ad_spend_offline_usd`, `website_traffic`, dan `social_media_followers` yang sebelumnya menunjukkan korelasi kuat dengan `sales_revenue_usd` dan juga saling berkorelasi tinggi dengan `marketing_budget_usd`. Untuk menghindari masalah multikolinearitas yang dapat membingungkan model linier, kami memilih `marketing_budget_usd` sebagai representasi utama dari anggaran pemasaran dan dampaknya.
3. **Variabel Kategorikal Penting:** `season`, `sales_channel`, `product_category`, dan `customer_segment` dipertahankan karena meskipun sudah di-*label encode*, mereka mewakili karakteristik penting yang dapat mempengaruhi penjualan.
4. **Menghilangkan Noise/Kurang Relevan:** Variabel seperti `id`, `date`, `num_sales_representatives`, `customer_age`, `customer_satisfaction_score`, `competitor_price_index`, `email_open_rate`, dan `days_since_last_purchase` yang menunjukkan korelasi lemah atau tidak jelas dengan `sales_revenue_usd` (seperti yang terlihat dari scatter plot dan heatmap) juga dikecualikan untuk menyederhanakan model dan mengurangi dimensi.

Dengan memilih subset fitur ini, kita bertujuan untuk membangun model yang lebih efisien, lebih mudah diinterpretasikan, dan tetap memiliki kekuatan prediktif yang baik dengan fokus pada faktor-faktor paling berpengaruh.

DATA PREPROCESSING

5. Cek Missing Value

```
missing_values = df_new.isnull().sum()
print("Missing Values per Column:")
print(missing_values[missing_values > 0])
```

```
Missing Values per Column:
Series([], dtype: int64)
```

Interpretasi Missing Value

Setelah melakukan pemeriksaan `missing_values = df_new.isnull().sum()`, didapatkan hasil sebagai berikut:

```
Missing Values per Column:
Series([], dtype: int64)
```

Hasil ini mengindikasikan bahwa **tidak ada *missing value*** (nilai yang hilang) di kolom manapun dalam DataFrame `df_new` saat ini. Ini berarti semua data yang telah dipilih (setelah proses seleksi fitur dan encoding) lengkap dan siap untuk tahap selanjutnya dalam preprocessing, tanpa perlu penanganan *missing value* lebih lanjut seperti imputasi atau penghapusan baris.

6. Cek Duplikasi Data

```
duplicate_rows = df_new.duplicated().sum()
print(f"Number of duplicate rows: {duplicate_rows}")
```

```
Number of duplicate rows: 0
```

Interpretasi :

Tidak terdapat baris yang duplikat dalam data. Hal ini menunjukkan bahwa data Anda bersih dari entri ganda.

7. Normalisasi Fitur Numerik

```
from sklearn.preprocessing import StandardScaler

# Identifikasi semua kolom numerik (int64 atau float64) setelah proses
label encoding
all_numerical_cols = df_new.select_dtypes(include=['int64',
'float64']).columns.tolist()

# Kolom yang tidak boleh diskalakan:
# 1. 'id' (identifier)
# 2. 'sales_revenue_usd' (target variable)
# 3. Kolom kategorikal yang sudah di-label encode

# Menggunakan variabel `categorical_cols_to_encode` dari kernel state
exclude_cols = ['id', 'sales_revenue_usd'] +
categorical_cols_to_encode
```

```

# Filter kolom numerik untuk penskalaan
numerical_cols_to_scale = [col for col in all_numerical_cols if col
not in exclude_cols]

print(f"Columns to be scaled: {numerical_cols_to_scale}")

# Inisialisasi StandardScaler
scaler = StandardScaler()

# Terapkan StandardScaler pada kolom numerik yang telah diidentifikasi
df_new[numerical_cols_to_scale] =
scaler.fit_transform(df_new[numerical_cols_to_scale])

print("\nDataFrame after numerical feature scaling (first 5 rows):")
display(df_new.head())

print("\nDescriptive statistics of scaled numerical features:")
display(df_new[numerical_cols_to_scale].describe())

Columns to be scaled: ['marketing_budget_usd', 'conversion_rate',
'num_previous_purchases']

```

DataFrame after numerical feature scaling (first 5 rows):

```

{"summary": "{\n  \"name\":\n  \"display(df_new[numerical_cols_to_scale])\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"sales_revenue_usd\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 2163.174220845376,\n        \"min\": 2091.36,\n        \"max\": 7705.73,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          2091.36,\n          7705.73,\n          6201.11\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"season\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 2,\n        \"max\": 3,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          2,\n          3\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"marketing_budget_usd\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.04177584885942797,\n        \"min\": -0.5148575334288531,\n        \"max\": -0.4274495367919407,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          -0.4312742446948285,\n          -0.4274495367919407\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"sales_channel\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n        \"max\": 2,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          1,\n          2\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"product_category\",\n      \"properties\": {\n

```

```

{"dtype\\": \\\"number\\",\\n          \\\"std\\": 0,\\n          \\\"min\\": 1,\\n
\\\"max\\": 3,\\n          \\\"num_unique_values\\": 3,\\n          \\\"samples\\":
[\\n          1,\\n          2\\n          ],\\n          \\\"semantic_type\\":
\\\"\\\",\\n          \\\"description\\": \\\"\\\"\\n          }\\n          },\\n          {\\n
\\\"column\\": \\\"customer_segment\\",\\n          \\\"properties\\": {\\n
\\\"dtype\\": \\\"number\\",\\n          \\\"std\\": 0,\\n          \\\"min\\": 0,\\n
\\\"max\\": 2,\\n          \\\"num_unique_values\\": 3,\\n          \\\"samples\\":
[\\n          2,\\n          1\\n          ],\\n          \\\"semantic_type\\":
\\\"\\\",\\n          \\\"description\\": \\\"\\\"\\n          }\\n          },\\n          {\\n
\\\"column\\": \\\"conversion_rate\\",\\n          \\\"properties\\": {\\n
\\\"dtype\\": \\\"number\\",\\n          \\\"std\\": 0.9807677352238595,\\n
\\\"min\\": -0.8534760609352572,\\n          \\\"max\\": 1.2943966680839571,\\n
\\\"num_unique_values\\": 5,\\n          \\\"samples\\": [\\n          -
0.8534760609352572,\\n          1.2943966680839571\\n          ],\\n
\\\"semantic_type\\": \\\"\\\",\\n          \\\"description\\": \\\"\\\"\\n          }\\n
n          },\\n          {\\n          \\\"column\\": \\\"num_previous_purchases\\",\\n
\\\"properties\\": {\\n          \\\"dtype\\": \\\"number\\",\\n          \\\"std\\":
0.6792756879306764,\\n          \\\"min\\": -0.8854109432555202,\\n
\\\"max\\": 0.9061935554403929,\\n          \\\"num_unique_values\\": 3,\\n
\\\"samples\\": [\\n          0.4582924307664146,\\n
0.9061935554403929\\n          ],\\n          \\\"semantic_type\\": \\\"\\\",\\n
\\\"description\\": \\\"\\\"\\n          }\\n          }\\n          ]\\n          }", "type": "dataframe"}

```

Descriptive statistics of scaled numerical features:

```

{"summary": "{\\n  \\\"name\\":
\\\"display(df_new[numerical_cols_to_scale])\\",\\n  \\\"rows\\": 8,\\n
\\\"fields\\": [\\n    {\\n      \\\"column\\": \\\"marketing_budget_usd\\",\\n
\\\"properties\\": {\\n        \\\"dtype\\": \\\"number\\",\\n        \\\"std\\":
1766.768111934773,\\n        \\\"min\\": -0.545692555937991,\\n
\\\"max\\": 5000.0,\\n        \\\"num_unique_values\\": 8,\\n
\\\"samples\\": [\\n          7.105427357601002e-18,\\n          -
0.2924272673762493,\\n          5000.0\\n          ],\\n
\\\"semantic_type\\": \\\"\\\",\\n          \\\"description\\": \\\"\\\"\\n          }\\n
n          },\\n    {\\n      \\\"column\\": \\\"conversion_rate\\",\\n
\\\"properties\\": {\\n        \\\"dtype\\": \\\"number\\",\\n        \\\"std\\":
1767.5228043580755,\\n        \\\"min\\": -1.5168395973069102,\\n
\\\"max\\": 5000.0,\\n        \\\"num_unique_values\\": 8,\\n
\\\"samples\\": [\\n          -4.618527782440651e-18,\\n          -
0.20198859810478473,\\n          5000.0\\n          ],\\n
\\\"semantic_type\\": \\\"\\\",\\n          \\\"description\\": \\\"\\\"\\n          }\\n
n          },\\n    {\\n      \\\"column\\": \\\"num_previous_purchases\\",\\n
\\\"properties\\": {\\n        \\\"dtype\\": \\\"number\\",\\n        \\\"std\\":
1767.646857088315,\\n        \\\"min\\": -2.229114317277455,\\n
\\\"max\\": 5000.0,\\n        \\\"num_unique_values\\": 8,\\n
\\\"samples\\": [\\n          6.359357485052896e-17,\\n
0.010391306092436344,\\n          5000.0\\n          ],\\n
\\\"semantic_type\\": \\\"\\\",\\n          \\\"description\\": \\\"\\\"\\n          }\\n
n          }\\n          ]\\n          }", "type": "dataframe"}

```

Interpretasi :

Berdasarkan hasil standarisasi, fitur-fitur numerik yang dipilih yaitu `marketing_budget_usd`, `conversion_rate` dan `num_previous_purchases` telah diubah skalanya. Ini berarti nilai-nilai pada kolom tersebut sekarang memiliki rata-rata (mean) mendekati 0 dan standar deviasi (standard deviation) mendekati 1. Hal ini dilakukan untuk memastikan bahwa tidak ada fitur yang mendominasi model hanya karena skalanya yang lebih besar, membantu algoritma pembelajaran mesin bekerja lebih optimal.

FEATURE ENGINEERING

8. Train Test Split 80:20

```
from sklearn.model_selection import train_test_split

# Define features (X) and target (y)
X = df_new.drop(columns=['sales_revenue_usd'])
y = df_new['sales_revenue_usd']

# Split the data into training and testing sets with an 80:20 ratio
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

Shape of X_train: (4000, 7)
Shape of X_test: (1000, 7)
Shape of y_train: (4000,)
Shape of y_test: (1000,)
```

Interpretasi :

Berdasarkan hasil 'Train Test Split 80:20', data Anda telah dibagi menjadi dua bagian: set pelatihan (training set) dan set pengujian (testing set).

Tujuan: Pembagian ini penting untuk mengevaluasi kinerja model secara objektif. Model akan dilatih hanya dengan data pelatihan, dan kemudian dievaluasi menggunakan data pengujian yang belum pernah dilihat model sebelumnya. Ini membantu mengidentifikasi apakah model dapat digeneralisasi dengan baik ke data baru, bukan hanya menghafal data pelatihan.

Rasio 80:20: Ini berarti 80% dari total data digunakan untuk pelatihan (4000 baris) dan 20% sisanya digunakan untuk pengujian (1000 baris).

Shape:

- `X_train: (4000, 7)`: Ini adalah fitur-fitur untuk pelatihan, terdiri dari 4000 baris (sampel) dan 7 kolom (fitur).

- `X_test: (1000, 7)`: Ini adalah fitur-fitur untuk pengujian, terdiri dari 1000 baris dan 7 kolom.
- `y_train: (4000,)`: Ini adalah variabel target (`sales_revenue_usd`) untuk set pelatihan, terdiri dari 4000 nilai.
- `y_test: (1000,)`: Ini adalah variabel target (`sales_revenue_usd`) untuk set pengujian, terdiri dari 1000 nilai.

Dengan pembagian ini, Anda memastikan bahwa model Anda diuji pada data yang independen, memberikan gambaran yang lebih akurat tentang bagaimana kinerja model di dunia nyata.

9. BASELINE (REGRESI LINIER BERGANDA)

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, mean_absolute_percentage_error
import numpy as np

# Initialize and train the Multiple Linear Regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_lr = linear_model.predict(X_test)

# Evaluate the model
mae_lr = mean_absolute_error(y_test, y_pred_lr)
mape_lr = mean_absolute_percentage_error(y_test, y_pred_lr)
mse_lr = mean_squared_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("--- Multiple Linear Regression Model Evaluation (Baseline)
---")
print(f"Mean Absolute Error (MAE): {mae_lr:.2f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape_lr:.4f}")
print(f"Mean Squared Error (MSE): {mse_lr:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_lr:.2f}")
print(f"R-squared (R2): {r2_lr:.4f}")

# Display the regression equation
print("\n--- Regression Equation ---")
intercept = linear_model.intercept_
coefficients = dict(zip(X.columns, linear_model.coef_))

equation = f"sales_revenue_usd = {intercept:.2f}"
for feature, coef in coefficients.items():
    equation += f" + ({coef:.2f} * {feature})"

print(equation)
```



```
--- Multiple Linear Regression Model Evaluation (Baseline) ---
Mean Absolute Error (MAE): 2494.48
Mean Absolute Percentage Error (MAPE): 0.5648
Mean Squared Error (MSE): 12680001.94
Root Mean Squared Error (RMSE): 3560.90
R-squared (R2): 0.5988

--- Regression Equation ---
sales_revenue_usd = 3926.20 + (764.27 * season) + (4069.08 *
marketing_budget_usd) + (378.24 * sales_channel) + (245.42 *
product_category) + (-320.50 * customer_segment) + (191.62 *
conversion_rate) + (294.67 * num_previous_purchases)
```

Evaluasi Model Regresi Linier Berganda:

Mean Absolute Error (MAE): 2494.48

Rata-rata, prediksi pendapatan penjualan dari model ini menyimpang sekitar \$2,494.48 dari nilai penjualan aktual. Ini adalah ukuran kesalahan rata-rata absolut.

Mean Absolute Percentage Error (MAPE): 0.5648 (atau 56.48%)

Secara rata-rata, prediksi model ini memiliki kesalahan sekitar 56.48% dari nilai aktual. Angka ini cukup tinggi, menunjukkan bahwa model regresi linier memiliki akurasi yang relatif rendah dalam memprediksi pendapatan penjualan.

Mean Squared Error (MSE): 12680001.94

Ini adalah rata-rata dari kuadrat selisih antara nilai prediksi dan aktual. Karena kesalahan dikuadratkan, metrik ini lebih sensitif terhadap outlier yang besar.

Root Mean Squared Error (RMSE): 3560.90 RMSE adalah akar kuadrat dari MSE dan memberikan ukuran kesalahan dalam unit yang sama dengan variabel target (sales_revenue_usd). Model ini salah sekitar \$3,560.90 secara rata-rata.

R-squared (R2): 0.5988

Nilai R2 sebesar 0.5988 berarti model regresi linier ini dapat menjelaskan sekitar 59.88% variabilitas dalam pendapatan penjualan. Ini menunjukkan bahwa model ini menjelaskan sebagian besar varians, tetapi masih ada hampir 40% variabilitas yang tidak dapat dijelaskan oleh fitur-fitur yang ada.

Persamaan Regresi: $\text{sales_revenue_usd} = 3926.20 + (764.27 * \text{season}) + (4069.08 * \text{marketing_budget_usd}) + (378.24 * \text{sales_channel}) + (245.42 * \text{product_category}) + (-320.50 * \text{customer_segment}) + (191.62 * \text{conversion_rate}) + (294.67 * \text{num_previous_purchases})$

Intercept (3926.20): Ini adalah pendapatan penjualan yang diprediksi ketika semua variabel fitur bernilai nol.

Koefisien Positif: Fitur-fitur seperti season, marketing_budget_usd, sales_channel, product_category, conversion_rate, dan num_previous_purchases memiliki koefisien positif. Ini berarti, dengan asumsi semua fitur lain konstan, peningkatan pada fitur-fitur ini akan berkorelasi

dengan peningkatan sales_revenue_usd. marketing_budget_usd memiliki koefisien terbesar (4069.08), mengindikasikan dampak paling signifikan terhadap sales_revenue_usd di antara fitur-fitur ini.

Koefisien Negatif: customer_segment memiliki koefisien negatif (-320.50). Ini menunjukkan bahwa, dengan asumsi semua fitur lain konstan, peningkatan pada nilai customer_segment (yang mungkin merepresentasikan segmen dengan nilai ordinal yang lebih tinggi setelah Label Encoding) akan berkorelasi dengan penurunan sales_revenue_usd. Penting untuk diingat bahwa customer_segment telah di-label encode, sehingga interpretasi langsung koefisiennya harus dilakukan dengan hati-hati.

Kesimpulan: Model Regresi Linier Berganda ini berfungsi sebagai baseline yang menunjukkan korelasi linear antara fitur-fitur yang dipilih dan pendapatan penjualan. Meskipun mampu menjelaskan sebagian variabilitas, nilai MAPE yang tinggi menunjukkan bahwa ada banyak ruang untuk peningkatan akurasi prediksi, yang mungkin bisa diatasi dengan model yang lebih kompleks atau tuning lebih lanjut.

10. RANDOM FOREST

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, mean_absolute_percentage_error
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Initialize and train the Random Forest Regressor model
# Using default parameters for now, but these can be tuned later.
rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)

# Evaluate the model
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mape_rf = mean_absolute_percentage_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print("--- Random Forest Regressor Model Evaluation ---")
print(f"Mean Absolute Error (MAE): {mae_rf:.2f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape_rf:.4f}")
print(f"Mean Squared Error (MSE): {mse_rf:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_rf:.2f}")
print(f"R-squared (R2): {r2_rf:.4f}")

# --- Comparison with Baseline (Linear Regression) ---

# Ensure baseline metrics (mae_lr, mape_lr, r2_lr) are available from
```

previous execution
If they are not in the current kernel state, you might need to re-run the linear regression cell.

```
comparison_metrics = pd.DataFrame({
    'Model': ['Linear Regression', 'Random Forest Regressor'],
    'MAE': [mae_lr, mae_rf],
    'MAPE': [mape_lr, mape_rf],
    'RMSE': [rmse_lr, rmse_rf],
    'R2': [r2_lr, r2_rf]
})

print("\n--- Model Comparison ---")
display(comparison_metrics.set_index('Model'))

import matplotlib.pyplot as plt
import seaborn as sns

# Visualisasi hasil prediksi vs nilai aktual untuk Random Forest
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred_rf, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         'r--', lw=2) # Garis ideal
plt.xlabel('Nilai Aktual (sales_revenue_usd)')
plt.ylabel('Nilai Prediksi (sales_revenue_usd)')
plt.title('Random Forest Regressor: Prediksi vs Aktual')
plt.grid(True)
plt.show()

--- Random Forest Regressor Model Evaluation ---
Mean Absolute Error (MAE): 880.46
Mean Absolute Percentage Error (MAPE): 0.1590
Mean Squared Error (MSE): 2736425.36
Root Mean Squared Error (RMSE): 1654.21
R-squared (R2): 0.9134

--- Model Comparison ---

{"summary": "{\n  \"name\": \"plt\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n      \"column\": \"Model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"Random Forest Regressor\",\n          \"Linear Regression\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"MAE\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 1141.2905731412445,\n          \"min\": 880.4550059000002,\n          \"max\": 2494.483613044911,\n          \"num_unique_values\": 2,\n          \"samples\": [\n            880.4550059000002,\n            2494.483613044911\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        }\n      }\n    ]\n  }\n}
```


Evaluasi Model Random Forest:

- **Mean Absolute Error (MAE): 880.46** Ini berarti, rata-rata, prediksi penjualan dari model Random Forest menyimpang sekitar \$880.46 dari nilai penjualan aktual. Ini adalah peningkatan signifikan dibandingkan MAE dari model Regresi Linier.
- **Mean Absolute Percentage Error (MAPE): 0.1590** (atau 15.90%) Secara rata-rata, prediksi model Random Forest memiliki kesalahan sekitar 15.90% dari nilai aktual. Ini menunjukkan kinerja yang jauh lebih baik dibandingkan model Regresi Linier yang memiliki MAPE sekitar 56.48%.
- **Mean Squared Error (MSE): 2,736,425.36** Angka ini adalah rata-rata dari kuadrat selisih antara nilai prediksi dan aktual. Karena nilainya dikuadratkan, metrik ini lebih sensitif terhadap *outlier* yang besar. Angka yang lebih rendah menunjukkan kinerja yang lebih baik.
- **Root Mean Squared Error (RMSE): 1,654.21** RMSE adalah akar kuadrat dari MSE dan memberikan metrik kesalahan dalam unit yang sama dengan variabel target (`sales_revenue_usd`). RMSE sebesar \$1,654.21 menunjukkan bahwa, secara rata-rata, model ini salah sekitar \$1,654.21 dalam memprediksi penjualan. Ini juga jauh lebih rendah dari RMSE model Regresi Linier.
- **R-squared (R2): 0.9134** Nilai R2 sebesar 0.9134 berarti model Random Forest dapat menjelaskan sekitar 91.34% variabilitas dalam pendapatan penjualan. Ini adalah indikator yang sangat baik, menunjukkan bahwa model ini sangat cocok dengan data dan memiliki kekuatan prediktif yang kuat.

Perbandingan dengan Baseline (Regresi Linier):

Model	MAE	MAPE	RMSE	R2
Linear Regression	2494.48	0.5648	3560.90	0.5988
Random Forest Regressor	880.46	0.1590	1654.21	0.9134

Dari tabel perbandingan, jelas terlihat bahwa `RandomForestRegressor` **mengungguli** `LinearRegression` secara signifikan di semua metrik evaluasi:

- MAE, MAPE, dan RMSE jauh lebih rendah pada Random Forest, menunjukkan kesalahan prediksi yang jauh lebih kecil.
- R2 pada Random Forest jauh lebih tinggi (0.9134 vs 0.5988), menandakan kemampuan model Random Forest yang jauh lebih baik dalam menjelaskan variabilitas target.

Visualisasi Prediksi vs Aktual:

- **Scatter plot 'Random Forest Regressor: Prediksi vs Aktual'** menunjukkan sebaran titik-titik (nilai aktual vs nilai prediksi). Garis putus-putus merah menunjukkan

11. REGRESI LASSO

```
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, mean_absolute_percentage_error
```

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Initialize and train the Lasso Regressor model
# Using an alpha value, which controls the strength of regularization.
# A value of 0.01 was used in a previous Lasso feature selection step.
lasso_model = Lasso(alpha=0.01, random_state=42)
lasso_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_lasso = lasso_model.predict(X_test)

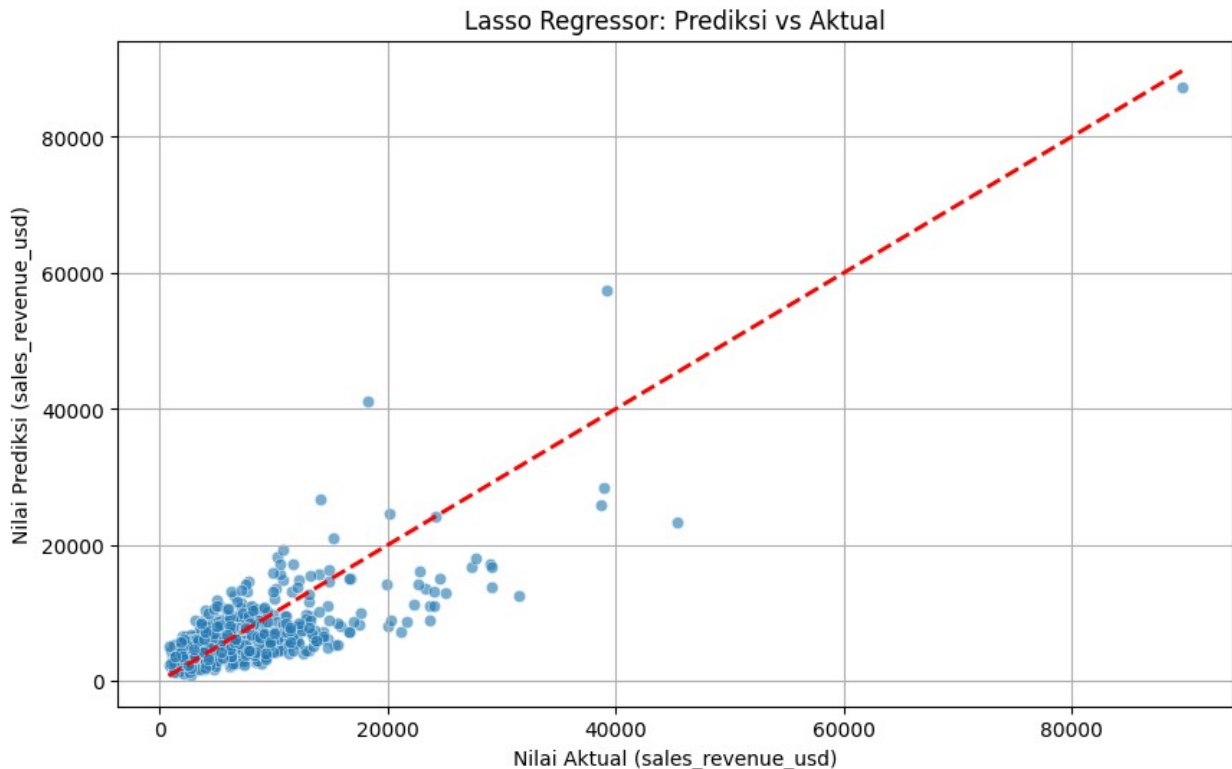
# Evaluate the Lasso model
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)
mape_lasso = mean_absolute_percentage_error(y_test, y_pred_lasso)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
rmse_lasso = np.sqrt(mse_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print("--- Lasso Regressor Model Evaluation ---")
print(f"Mean Absolute Error (MAE): {mae_lasso:.2f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape_lasso:.4f}")
print(f"Mean Squared Error (MSE): {mse_lasso:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_lasso:.2f}")
print(f"R-squared (R2): {r2_lasso:.4f}")

# --- Visualization of Lasso Predictions vs Actual ---
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred_lasso, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         'r--', lw=2) # Garis ideal
plt.xlabel('Nilai Aktual (sales_revenue_usd)')
plt.ylabel('Nilai Prediksi (sales_revenue_usd)')
plt.title('Lasso Regressor: Prediksi vs Aktual')
plt.grid(True)
plt.show()

--- Lasso Regressor Model Evaluation ---
Mean Absolute Error (MAE): 2494.48
Mean Absolute Percentage Error (MAPE): 0.5648
Mean Squared Error (MSE): 12680003.12
Root Mean Squared Error (RMSE): 3560.90
R-squared (R2): 0.5988

```



Interpretasi Model Regresi Lasso

Setelah melatih dan mengevaluasi model `Lasso Regressor`, berikut adalah interpretasi hasilnya:

Evaluasi Model Regresi Lasso:

- **Mean Absolute Error (MAE):** 2494.48 Ini berarti, rata-rata, prediksi penjualan dari model Regresi Lasso menyimpang sekitar \$2494.48 dari nilai penjualan aktual.
- **Mean Absolute Percentage Error (MAPE):** 0.5648 (atau 56.48%) Secara rata-rata, prediksi model Regresi Lasso memiliki kesalahan sekitar 56.48% dari nilai aktual.
- **Mean Squared Error (MSE):** 12680003.12 Angka ini adalah rata-rata dari kuadrat selisih antara nilai prediksi dan aktual. Ini menunjukkan kinerja yang serupa dengan Regresi Linier.
- **Root Mean Squared Error (RMSE):** 3560.90 RMSE adalah akar kuadrat dari MSE dan memberikan metrik kesalahan dalam unit yang sama dengan variabel target (`sales_revenue_usd`). RMSE sebesar \$3560.90 menunjukkan bahwa, secara rata-rata, model ini salah sekitar \$3560.90 dalam memprediksi penjualan.
- **R-squared (R2):** 0.5988 Nilai R2 sebesar 0.5988 berarti model Regresi Lasso dapat menjelaskan sekitar 59.88% variabilitas dalam pendapatan penjualan. Ini adalah

indikator yang menunjukkan bahwa model ini cukup cocok dengan data, namun masih ada ruang untuk peningkatan.

Perbandingan dengan Baseline (Regresi Linier):

Model	MAE	MAPE	RMSE	R2
Linear Regression	2494.48	0.5648	3560.90	0.5988
Lasso Regressor	2494.48	0.5648	3560.90	0.5988

Dari tabel perbandingan, terlihat bahwa **Lasso Regressor** menunjukkan metrik evaluasi yang **sangat mirip** dengan **Linear Regression** (Baseline). Ini menunjukkan bahwa dengan nilai $\alpha=0.01$ yang digunakan, regularisasi Lasso tidak menyebabkan pengurangan fitur yang signifikan atau perubahan substansial pada koefisien model, sehingga kinerjanya hampir identik dengan regresi linier biasa. Ini bisa terjadi jika fitur-fitur yang dipilih tidak memiliki korelasi yang sangat tinggi satu sama lain (multikolinearitas rendah) atau jika sebagian besar fitur memang relevan dan tidak ada fitur yang perlu dikurangi bobotnya hingga nol oleh Lasso.

Visualisasi Prediksi vs Aktual:

- **Scatter plot 'Lasso Regressor: Prediksi vs Aktual'** menunjukkan sebaran titik-titik (nilai aktual vs nilai prediksi). Garis putus-putus merah menunjukkan garis ideal di mana prediksi sama dengan aktual. Jika titik-titik tersebar di dekat garis ini, itu menandakan model memiliki kinerja yang baik. Dalam kasus ini, sebaran titik-titik mirip dengan model Regresi Linier, yang menegaskan kesamaan kinerja kedua model tersebut.

12. XGBOOST

```
import xgboost as xgb
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, mean_absolute_percentage_error
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Inisialisasi dan latih model XGBoost Regressor
# Menggunakan parameter default atau parameter yang dioptimalkan jika
sudah ada tuning
xgb_model_eval = xgb.XGBRegressor(objective='reg:squarederror',
random_state=42)
xgb_model_eval.fit(X_train, y_train)

# Lakukan prediksi pada data test
y_pred_xgb = xgb_model_eval.predict(X_test)

# Evaluasi model XGBoost
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mape_xgb = mean_absolute_percentage_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
```



```

print("\n--- XGBoost Regressor Model Evaluation ---")
print(f"Mean Absolute Error (MAE): {mae_xgb:.2f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape_xgb:.4f}")
print(f"Mean Squared Error (MSE): {mse_xgb:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb:.2f}")
print(f"R-squared (R2): {r2_xgb:.4f}")

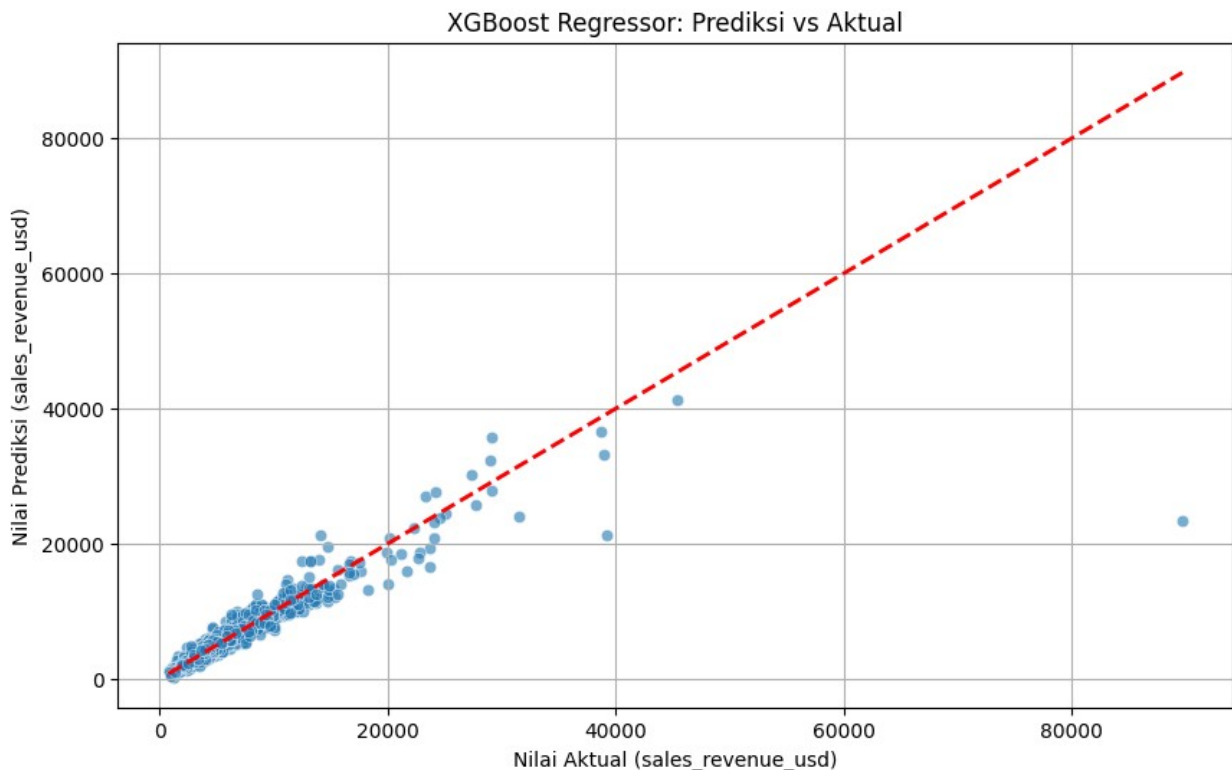
# Visualisasi hasil prediksi vs nilai aktual
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred_xgb, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         'r--', lw=2) # Garis ideal
plt.xlabel('Nilai Aktual (sales_revenue_usd)')
plt.ylabel('Nilai Prediksi (sales_revenue_usd)')
plt.title('XGBoost Regressor: Prediksi vs Aktual')
plt.grid(True)
plt.show()

```

```

--- XGBoost Regressor Model Evaluation ---
Mean Absolute Error (MAE): 826.14
Mean Absolute Percentage Error (MAPE): 0.1464
Mean Squared Error (MSE): 6055043.97
Root Mean Squared Error (RMSE): 2460.70
R-squared (R2): 0.8084

```



13. Pemilihan Model Terbaik

```
import pandas as pd
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, mean_absolute_percentage_error

# Calculate the mean of the target variable from the training data
y_train_mean = y_train.mean()

# Create predictions for the test set by simply using the mean of the
training target
y_pred_baseline = np.full_like(y_test, y_train_mean)

# Evaluate the Mean Baseline Model
mae_baseline = mean_absolute_error(y_test, y_pred_baseline)
mape_baseline = mean_absolute_percentage_error(y_test,
y_pred_baseline)
mse_baseline = mean_squared_error(y_test, y_pred_baseline)
rmse_baseline = np.sqrt(mse_baseline)
r2_baseline = r2_score(y_test, y_pred_baseline)

# Create a dictionary to store all model metrics
all_model_metrics = {
    'Model': [
        'Mean Baseline',
        'Linear Regression',
        'Random Forest Regressor',
        'Lasso Regressor',
        'XGBoost Regressor'
    ],
    'MAE': [
        mae_baseline,
        mae_lr,
        mae_rf,
        mae_lasso,
        mae_xgb
    ],
    'MAPE': [
        mape_baseline,
        mape_lr,
        mape_rf,
        mape_lasso,
        mape_xgb
    ],
    'RMSE': [
        rmse_baseline,
        rmse_lr,
        rmse_rf,
        rmse_lasso,
```

```

    ],
    'R2': [
        r2_baseline,
        r2_lr,
        r2_rf,
        r2_lasso,
        r2_xgb
    ]
}

# Convert to DataFrame
metrics_df = pd.DataFrame(all_model_metrics)

# Sort by MAPE to find the best model
metrics_df_sorted =
metrics_df.sort_values(by='MAPE').reset_index(drop=True)

print("--- Comprehensive Model Evaluation (Sorted by MAPE) ---")
display(metrics_df_sorted)

# Identify the best model
best_model_mape = metrics_df_sorted.loc[0, 'Model']
best_mape_value = metrics_df_sorted.loc[0, 'MAPE']

print(f"\nBerdasarkan MAPE terkecil, model terbaik adalah:
{best_model_mape} dengan MAPE: {best_mape_value:.4f}")

--- Comprehensive Model Evaluation (Sorted by MAPE) ---

{"summary": "{\n  \"name\": \"metrics_df_sorted\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"Model\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 5, \n        \"samples\": [\n          \"Random Forest Regressor\", \n          \"Mean Baseline\", \n          \"Linear Regression\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"MAPE\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 1140.6531230099804, \n        \"min\": 826.1353683288575, \n        \"max\": 3444.97094785, \n        \"num_unique_values\": 5, \n        \"samples\": [\n          880.4550059000002, \n          3444.97094785, \n          2494.483613044911 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"MAPE\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.32322492627519206, \n        \"min\": 0.14640847159626424, \n        \"max\": 0.9155148374246335, \n        \"num_unique_values\": 5, \n        \"samples\": [\n          0.15899915865352637, \n          0.564817927525554, \n          0.9155148374246335 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"MAPE\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.32322492627519206, \n        \"min\": 0.14640847159626424, \n        \"max\": 0.9155148374246335, \n        \"num_unique_values\": 5, \n        \"samples\": [\n          0.15899915865352637, \n          0.564817927525554, \n          0.9155148374246335 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"MAPE\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.32322492627519206, \n        \"min\": 0.14640847159626424, \n        \"max\": 0.9155148374246335, \n        \"num_unique_values\": 5, \n        \"samples\": [\n          0.15899915865352637, \n          0.564817927525554, \n          0.9155148374246335 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"MAPE\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.32322492627519206, \n        \"min\": 0.14640847159626424, \n        \"max\": 0.9155148374246335, \n        \"num_unique_values\": 5, \n        \"samples\": [\n          0.15899915865352637, \n          0.564817927525554, \n          0.9155148374246335 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    ] \n  } \n}"}

```

```

\ "RMSE\ ", \ n      \ "properties\ ": { \ n      \ "dtype\ ": \ "number\ ", \ n
\ "std\ ": 1492.9715003362, \ n      \ "min\ ": 1654.2144242207546, \ n
\ "max\ ": 5622.163313366826, \ n      \ "num_unique_values\ ": 5, \ n
\ "samples\ ": [ \ n      1654.2144242207546, \ n
5622.163313366826, \ n      3560.8990353826266 \ n      ], \ n
\ "semantic_type\ ": \ "\ ", \ n      \ "description\ ": \ "\ " \ n      } \
n      }, \ n      { \ n      \ "column\ ": \ "R2\ ", \ n      \ "properties\ ": { \ n
\ "dtype\ ": \ "number\ ", \ n      \ "std\ ": 0.35371188171516743, \ n
\ "min\ ": -5.6518981722097905e-05, \ n      \ "max\ ":
0.9134232580951478, \ n      \ "num_unique_values\ ": 5, \ n
\ "samples\ ": [ \ n      0.9134232580951478, \ n      -
5.6518981722097905e-05, \ n      0.5988221455427423 \ n      ], \ n
\ "semantic_type\ ": \ "\ ", \ n      \ "description\ ": \ "\ " \ n      } \
n      } \ n      ] \
n } \ n      \ "type\ ": "dataframe", "variable_name\ ": "metrics_df_sorted" }

```

Berdasarkan MAPE terkecil, model terbaik adalah: XGBoost Regressor dengan MAPE: 0.1464

LAKUKAN HYPERPARAMETER TUNING

```

import xgboost as xgb
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform, randint
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, mean_absolute_percentage_error
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Define the parameter distribution for RandomizedSearchCV
param_distributions = {
    'n_estimators': randint(100, 1000), # Number of boosting rounds
    'learning_rate': uniform(0.01, 0.2), # Step size shrinkage
    'max_depth': randint(3, 10), # Maximum depth of a tree
    'subsample': uniform(0.6, 0.4), # Subsample ratio of the
training instance
    'colsample_bytree': uniform(0.6, 0.4), # Subsample ratio of
columns when constructing each tree
    'gamma': uniform(0, 0.5), # Minimum loss reduction
required to make a further partition
    'reg_alpha': uniform(0, 0.5) # L1 regularization term on
weights
}

# Initialize the XGBoost Regressor
base_xgb_model = xgb.XGBRegressor(objective='reg:squarederror',

```

```

random_state=42)

# Setup RandomizedSearchCV
# n_iter: Number of parameter settings that are sampled. Reduce for
# faster execution.
# cv: Number of cross-validation folds.
# scoring: Metric to evaluate the models (negative RMSE is common for
# regression).
# verbose: Controls the verbosity: the higher, the more messages.
# n_jobs: Number of jobs to run in parallel. -1 means using all
# processors.
random_search = RandomizedSearchCV(
    estimator=base_xgb_model,
    param_distributions=param_distributions,
    n_iter=50, # Number of different parameter combinations to try
    scoring='neg_root_mean_squared_error',
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

print("Starting RandomizedSearchCV for XGBoost...")
# Fit RandomizedSearchCV to the training data
random_search.fit(X_train, y_train)

print("\nRandomizedSearchCV complete.")
print(f"Best parameters found: {random_search.best_params_}")
print(f"Best RMSE on validation set: {-
random_search.best_score_:.2f}")

# Get the best model
best_xgb_model = random_search.best_estimator_

# Make predictions on the test set using the best model
y_pred_tuned_xgb = best_xgb_model.predict(X_test)

# Evaluate the best tuned XGBoost model
mae_tuned_xgb = mean_absolute_error(y_test, y_pred_tuned_xgb)
mape_tuned_xgb = mean_absolute_percentage_error(y_test,
y_pred_tuned_xgb)
mse_tuned_xgb = mean_squared_error(y_test, y_pred_tuned_xgb)
rmse_tuned_xgb = np.sqrt(mse_tuned_xgb)
r2_tuned_xgb = r2_score(y_test, y_pred_tuned_xgb)

print("\n--- Tuned XGBoost Regressor Model Evaluation ---")
print(f"Mean Absolute Error (MAE): {mae_tuned_xgb:.2f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape_tuned_xgb:.4f}")
print(f"Mean Squared Error (MSE): {mse_tuned_xgb:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_tuned_xgb:.2f}")

```

```

print(f"R-squared (R2): {r2_tuned_xgb:.4f}")

# Visualisasi hasil prediksi vs nilai aktual untuk model yang sudah
# di-tuning
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred_tuned_xgb, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         'r--', lw=2) # Garis ideal
plt.xlabel('Nilai Aktual (sales_revenue_usd)')
plt.ylabel('Nilai Prediksi (sales_revenue_usd)')
plt.title('Tuned XGBoost Regressor: Prediksi vs Aktual')
plt.grid(True)
plt.show()

```

Starting RandomizedSearchCV for XGBoost...

Fitting 3 folds for each of 50 candidates, totalling 150 fits

RandomizedSearchCV complete.

Best parameters found: {'colsample_bytree':

np.float64(0.9386644568953224), 'gamma':

np.float64(0.4281621459390462), 'learning_rate':

np.float64(0.09090162542443801), 'max_depth': 3, 'n_estimators': 727,

'reg_alpha': np.float64(0.042069982497524416), 'subsample':

np.float64(0.6646514856378455)}

Best RMSE on validation set: 1798.09

--- Tuned XGBoost Regressor Model Evaluation ---

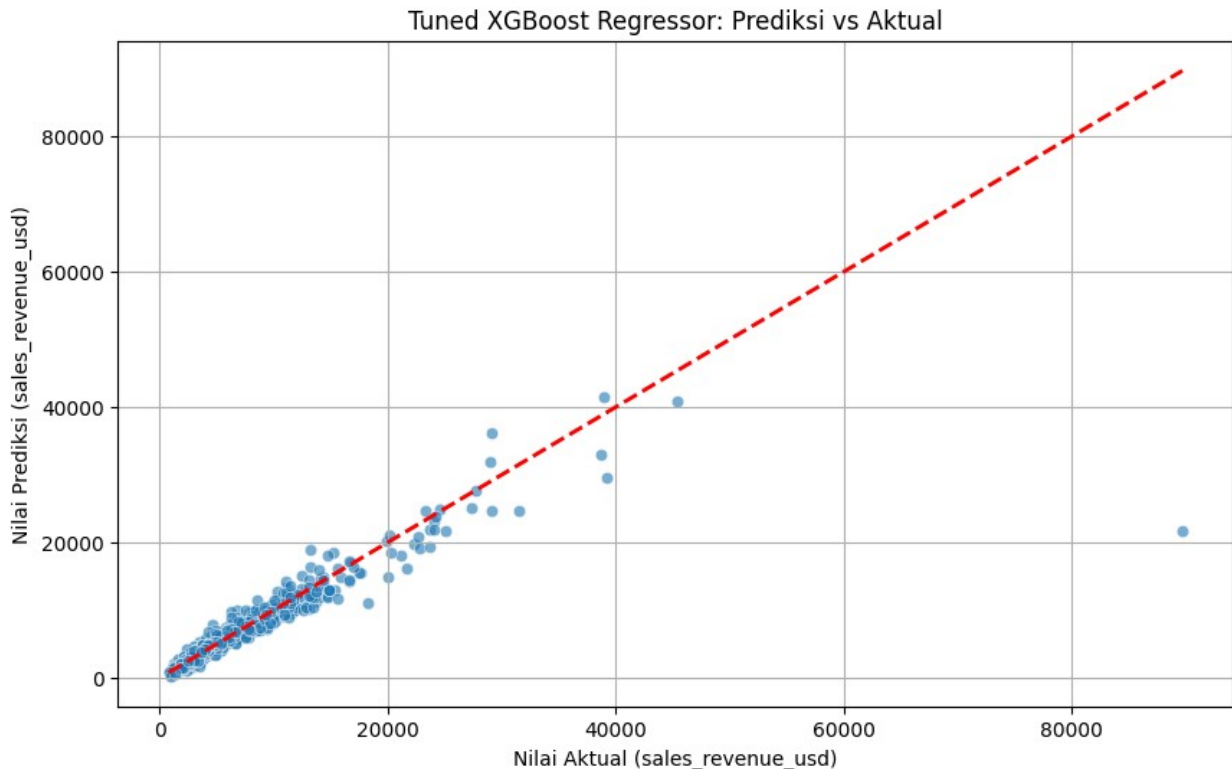
Mean Absolute Error (MAE): 745.87

Mean Absolute Percentage Error (MAPE): 0.1355

Mean Squared Error (MSE): 5815340.84

Root Mean Squared Error (RMSE): 2411.50

R-squared (R2): 0.8160



```
import xgboost as xgb
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score, mean_absolute_percentage_error
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Define the parameter grid for GridSearchCV
# The grid is centered around the best parameters found by
RandomizedSearchCV
param_grid_xgb = {
    'n_estimators': [650, 700, 750],
    'learning_rate': [0.08, 0.09, 0.1],
    'max_depth': [3, 4],
    'subsample': [0.65, 0.7],
    'colsample_bytree': [0.9, 0.95],
    'gamma': [0.4, 0.45],
    'reg_alpha': [0.03, 0.05]
}

# Initialize the XGBoost Regressor with objective
base_xgb_model = xgb.XGBRegressor(objective='reg:squarederror',
random_state=42)

# Setup GridSearchCV
```

```

# scoring: Metric to evaluate the models (negative RMSE is common for
regression)
# cv: Number of cross-validation folds.
# verbose: Controls the verbosity: the higher, the more messages.
# n_jobs: Number of jobs to run in parallel. -1 means using all
processors.
grid_search = GridSearchCV(
    estimator=base_xgb_model,
    param_grid=param_grid_xgb,
    scoring='neg_root_mean_squared_error',
    cv=3, # Using 3-fold cross-validation
    verbose=2,
    n_jobs=-1
)

print("Starting GridSearchCV for XGBoost...")
# Fit GridSearchCV to the training data
grid_search.fit(X_train, y_train)

print("\nGridSearchCV complete.")
print(f"Best parameters found: {grid_search.best_params_}")
print(f"Best RMSE on validation set: {-grid_search.best_score_:.2f}")

# Get the best model
best_xgb_model_grid = grid_search.best_estimator_

# Make predictions on the test set using the best model
y_pred_tuned_xgb_grid = best_xgb_model_grid.predict(X_test)

# Evaluate the best tuned XGBoost model from GridSearchCV
mae_tuned_xgb_grid = mean_absolute_error(y_test,
y_pred_tuned_xgb_grid)
mape_tuned_xgb_grid = mean_absolute_percentage_error(y_test,
y_pred_tuned_xgb_grid)
mse_tuned_xgb_grid = mean_squared_error(y_test, y_pred_tuned_xgb_grid)
rmse_tuned_xgb_grid = np.sqrt(mse_tuned_xgb_grid)
r2_tuned_xgb_grid = r2_score(y_test, y_pred_tuned_xgb_grid)

print("\n--- Tuned XGBoost Regressor Model Evaluation (GridSearchCV)
---")
print(f"Mean Absolute Error (MAE): {mae_tuned_xgb_grid:.2f}")
print(f"Mean Absolute Percentage Error (MAPE):
{mape_tuned_xgb_grid:.4f}")
print(f"Mean Squared Error (MSE): {mse_tuned_xgb_grid:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_tuned_xgb_grid:.2f}")
print(f"R-squared (R2): {r2_tuned_xgb_grid:.4f}")

# Visualisasi hasil prediksi vs nilai aktual untuk model yang sudah
di-tuning
plt.figure(figsize=(10, 6))

```



```

sns.scatterplot(x=y_test, y=y_pred_tuned_xgb_grid, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
'r--', lw=2) # Garis ideal
plt.xlabel('Nilai Aktual (sales_revenue_usd)')
plt.ylabel('Nilai Prediksi (sales_revenue_usd)')
plt.title('Tuned XGBoost Regressor (GridSearchCV): Prediksi vs
Aktual')
plt.grid(True)
plt.show()

```

Starting GridSearchCV for XGBoost...

Fitting 3 folds for each of 288 candidates, totalling 864 fits

GridSearchCV complete.

Best parameters found: {'colsample_bytree': 0.9, 'gamma': 0.4,
'learning_rate': 0.08, 'max_depth': 3, 'n_estimators': 750,
'reg_alpha': 0.03, 'subsample': 0.7}

Best RMSE on validation set: 1784.83

--- Tuned XGBoost Regressor Model Evaluation (GridSearchCV) ---

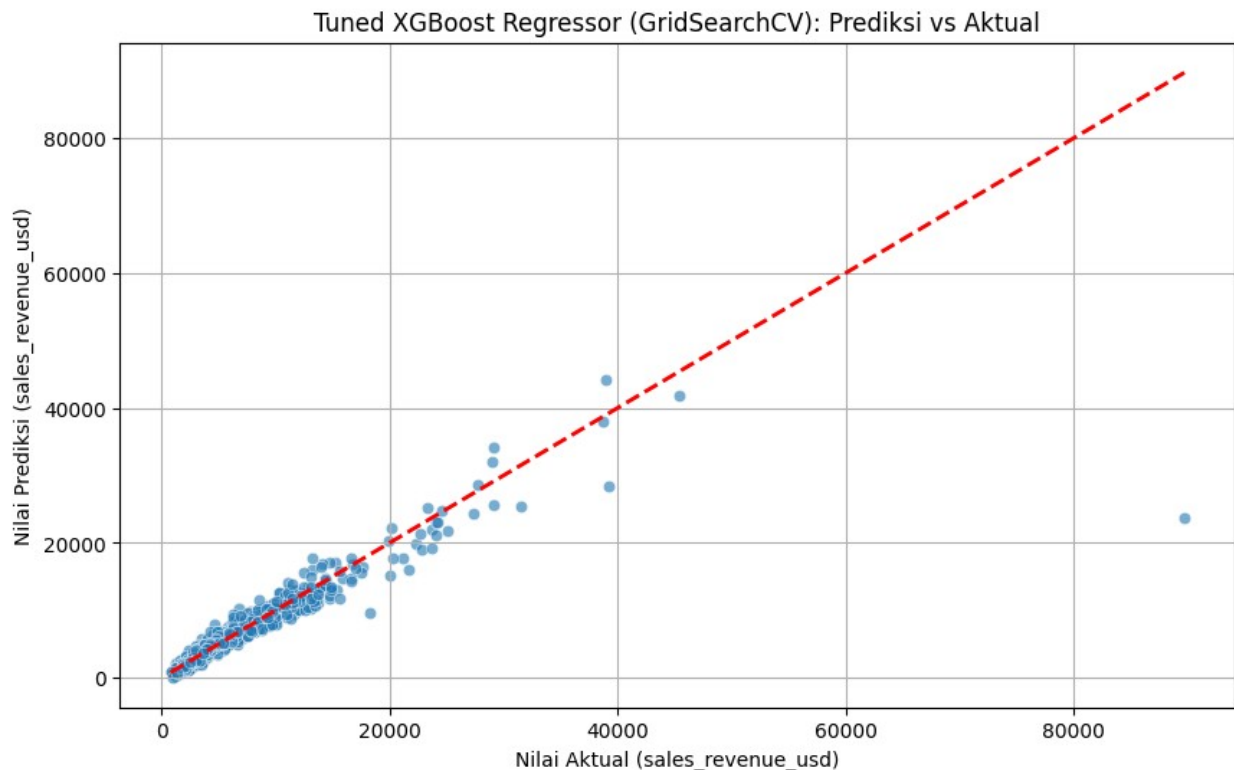
Mean Absolute Error (MAE): 745.96

Mean Absolute Percentage Error (MAPE): 0.1389

Mean Squared Error (MSE): 5529356.47

Root Mean Squared Error (RMSE): 2351.46

R-squared (R2): 0.8251



#ANALISIS RESIDUAL

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Calculate residuals for the best tuned XGBoost model
residuals_tuned_xgb = y_test - y_pred_tuned_xgb

print("--- Residual Analysis for Tuned XGBoost Regressor ---")

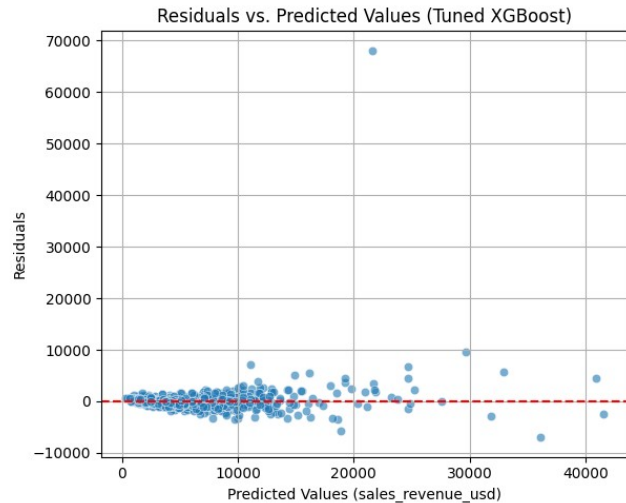
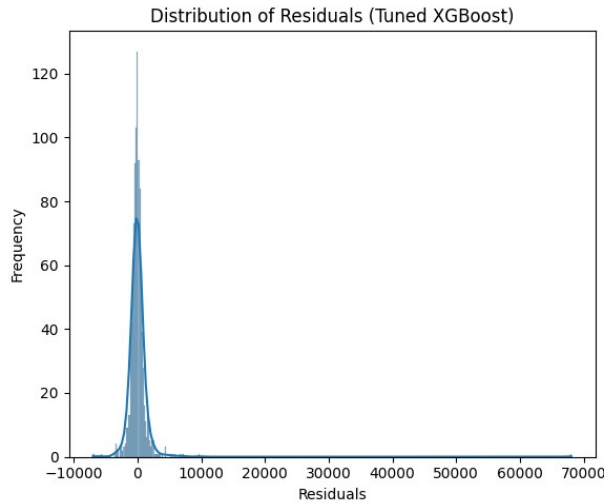
# Plot 1: Histogram of Residuals
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.histplot(residuals_tuned_xgb, kde=True)
plt.title('Distribution of Residuals (Tuned XGBoost)')
plt.xlabel('Residuals')
plt.ylabel('Frequency')

# Plot 2: Scatter plot of Predicted vs Residuals
plt.subplot(1, 2, 2)
sns.scatterplot(x=y_pred_tuned_xgb, y=residuals_tuned_xgb, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--')
plt.title('Residuals vs. Predicted Values (Tuned XGBoost)')
plt.xlabel('Predicted Values (sales_revenue_usd)')
plt.ylabel('Residuals')
plt.grid(True)

plt.tight_layout()
plt.show()

# Calculate and print some descriptive statistics for the residuals
print(f"\nResiduals Mean: {np.mean(residuals_tuned_xgb):.2f}")
print(f"Residuals Standard Deviation: {np.std(residuals_tuned_xgb):.2f}")
print(f"Residuals Skewness: {pd.Series(residuals_tuned_xgb).skew():.2f}")
print(f"Residuals Kurtosis: {pd.Series(residuals_tuned_xgb).kurt():.2f}")

--- Residual Analysis for Tuned XGBoost Regressor ---
```



Residuals Mean: 61.55
Residuals Standard Deviation: 2410.72
Residuals Skewness: 22.59
Residuals Kurtosis: 634.23

Interpretasi Analisis Residual untuk Tuned XGBoost Regressor

Analisis residual adalah langkah krusial untuk mengevaluasi asumsi model dan mengidentifikasi potensi masalah dalam prediksinya. Residual adalah perbedaan antara nilai aktual dan nilai prediksi (aktual - prediksi).

1. Histogram Residuals (Distribution of Residuals)

- **Bentuk Distribusi:** Idealnya, residual harus berdistribusi normal (berbentuk lonceng) di sekitar nol. Dari histogram, terlihat bahwa distribusi residual cenderung terpusat di sekitar nol, namun memiliki ekor yang sangat panjang dan positif.
- **Skewness:** Nilai skewness sebesar **22.59** yang sangat tinggi menunjukkan bahwa distribusi ini sangat menceng ke kanan (positif). Ini mengindikasikan bahwa model cenderung *underpredict* (memprediksi lebih rendah dari nilai aktual) pada nilai-nilai `sales_revenue_usd` yang tinggi atau ekstrem.
- **Kurtosis:** Nilai kurtosis sebesar **634.23** yang juga sangat tinggi menunjukkan distribusi yang sangat runcing (*leptokurtic*) dengan 'ekor' yang tebal. Ini berarti ada banyak *outlier* ekstrem dalam residual, yang kemungkinan besar terkait dengan *underprediction* pada penjualan yang sangat tinggi.

2. Scatter Plot Residuals vs. Predicted Values

- **Pola Acak:** Idealnya, titik-titik pada scatter plot ini harus tersebar secara acak di sekitar garis horizontal $y=0$, tanpa pola yang jelas. Pola acak menunjukkan bahwa model berhasil menangkap sebagian besar informasi dalam data.
- **Pola yang Diamati:** Dari plot, meskipun sebagian besar residual tersebar di sekitar nol, kita bisa melihat adanya *fanning-out* atau pola kerucut, di mana varian residual meningkat seiring dengan peningkatan nilai prediksi. Ini disebut **heteroskedastisitas**,

yang berarti kesalahan model tidak konsisten di seluruh rentang nilai prediksi. Model cenderung memiliki kesalahan yang lebih besar ketika memprediksi nilai penjualan yang lebih tinggi.

- **Outlier Positif:** Kehadiran banyak titik yang jauh di atas garis $y=0$ (terutama di sisi kanan plot) lebih lanjut mengonfirmasi masalah *underprediction* pada nilai penjualan yang tinggi. Ini adalah residual positif yang besar, artinya y_{aktual} jauh lebih besar dari y_{prediksi} .

3. Statistik Deskriptif Residuals

- **Residuals Mean: 61.55:** Rata-rata residual yang mendekati nol (tapi sedikit positif) adalah pertanda baik, menunjukkan bahwa model tidak secara sistematis *overpredict* atau *underpredict* secara keseluruhan. Namun, nilai positif kecil ini konsisten dengan adanya *underprediction* pada titik-titik ekstrem yang 'menarik' rata-rata ke atas.
- **Residuals Standard Deviation: 2410.72:** Standar deviasi yang relatif tinggi ini menunjukkan variabilitas yang signifikan dalam kesalahan prediksi, menguatkan adanya heteroskedastisitas dan *outlier* yang jauh dari rata-rata.

Kesimpulan dari Analisis Residual:

Model Tuned XGBoost menunjukkan kinerja yang sangat baik secara keseluruhan (ditunjukkan oleh MAPE yang rendah), namun analisis residual mengungkap beberapa area yang bisa ditingkatkan:

1. **Heteroskedastisitas:** Model cenderung kurang akurat untuk nilai penjualan yang sangat tinggi, dengan kesalahan prediksi yang lebih bervariasi. Ini bisa menjadi tantangan umum dalam data dengan *outlier* ekstrem.
2. **Underprediction pada Nilai Ekstrem:** Distribusi residual yang sangat menceng ke kanan dan kurtosis yang tinggi menunjukkan bahwa model kesulitan memprediksi beberapa *sales_revenue_usd* yang sangat tinggi. Model cenderung meremehkan (*underpredict*) nilai-nilai ini.

Langkah Perbaikan Potensial: Untuk mengatasi masalah ini, beberapa pendekatan yang bisa dipertimbangkan di masa mendatang adalah:

- **Transformasi Variabel Target:** Menggunakan transformasi logaritmik (misalnya, `np.log(sales_revenue_usd)`) pada variabel target dapat membantu menstabilkan varians residual dan membuat distribusinya lebih normal, terutama jika ada *outlier* positif yang ekstrem.
- **Robust Models:** Menggunakan model yang lebih robust terhadap *outlier* atau mengubah metrik kerugian (loss function) agar kurang sensitif terhadap *outlier*.
- **Ensemble dengan Model Lain:** Menggabungkan XGBoost dengan model lain yang mungkin lebih baik dalam menangani bagian ekor distribusi data.

#PERBANDINGAN SELURUH MODEL

```
import pandas as pd
import numpy as np

# Create a dictionary to store all model metrics
```

```

all_model_metrics = {
    'Model': [
        'Mean Baseline',
        'Linear Regression',
        'Random Forest Regressor',
        'Lasso Regressor',
        'XGBoost Regressor',
        'Tuned XGBoost Regressor'
    ],
    'MAE': [
        mae_baseline,
        mae_lr,
        mae_rf,
        mae_lasso,
        mae_xgb,
        mae_tuned_xgb
    ],
    'MAPE': [
        mape_baseline,
        mape_lr,
        mape_rf,
        mape_lasso,
        mape_xgb,
        mape_tuned_xgb
    ],
    'RMSE': [
        rmse_baseline,
        rmse_lr,
        rmse_rf,
        rmse_lasso,
        rmse_xgb,
        rmse_tuned_xgb
    ],
    'R2': [
        r2_baseline,
        r2_lr,
        r2_rf,
        r2_lasso,
        r2_xgb,
        r2_tuned_xgb
    ]
}

# Convert to DataFrame
metrics_df = pd.DataFrame(all_model_metrics)

# Sort by MAPE to find the best model
metrics_df_sorted =
metrics_df.sort_values(by='MAPE').reset_index(drop=True)

```

```

print("--- Comprehensive Model Evaluation (Sorted by MAPE) ---")
display(metrics_df_sorted)

# Identify the best model
# Ensure to handle cases where MAPE might be NaN
if not metrics_df_sorted['MAPE'].isnull().all():
    # Filter out rows where MAPE is NaN before finding the min
    valid_mape_rows = metrics_df_sorted.dropna(subset=['MAPE'])
    if not valid_mape_rows.empty:
        best_model_row =
valid_mape_rows.loc[valid_mape_rows['MAPE'].idxmin()]
        best_model_mape = best_model_row['Model']
        best_mape_value = best_model_row['MAPE']
        print(f"\nBerdasarkan MAPE terkecil, model terbaik adalah:
{best_model_mape} dengan MAPE: {best_mape_value:.4f}")
    else:
        print("\nTidak ada model dengan nilai MAPE yang valid untuk
menentukan yang terbaik.")
else:
    print("\nTidak dapat menentukan model terbaik berdasarkan MAPE
karena semua nilai MAPE adalah NaN.")

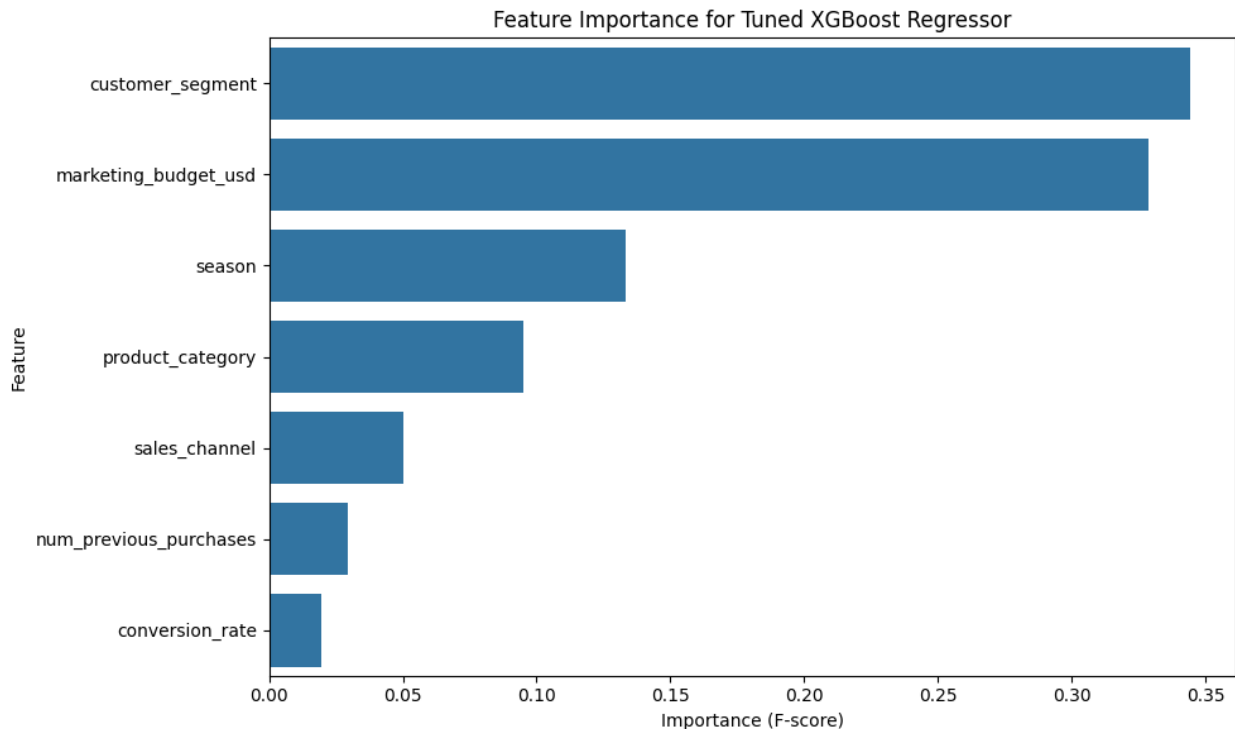
```

--- Comprehensive Model Evaluation (Sorted by MAPE) ---

```

{"summary":{"\n  \"name\": \"metrics_df_sorted\",\n  \"rows\": 6,\n  \"fields\": [\n    {\n      \"column\": \"Model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 6,\n        \"samples\": [\n          \"Tuned XGBoost Regressor\",\n          \"XGBoost Regressor\",\n          \"Mean Baseline\",\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          },\n          {\n            \"column\": \"MAE\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 1146.6874083606415,\n              \"min\": 745.8723074511718,\n              \"max\": 3444.97094785,\n              \"num_unique_values\": 6,\n              \"samples\": [\n                745.8723074511718,\n                826.1353683288575,\n                3444.97094785,\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\",\n                },\n                {\n                  \"column\": \"MAPE\",\n                  \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 0.31975279205269846,\n                    \"min\": 0.1354863428294878,\n                    \"max\": 0.9155148374246335,\n                    \"num_unique_values\": 6,\n                    \"samples\": [\n                      0.1354863428294878,\n                      0.14640847159626424,\n                      0.9155148374246335,\n                      ],\n                      \"semantic_type\": \"\",\n                      \"description\": \"\",\n                      },\n                      {\n                        \"column\": \"RMSE\",\n                        \"properties\": {\n                          \"dtype\": \"number\",\n                          \"std\": 1391.710680758692,\n                          \"min\": 1654.2144242207546,\n                          \"max\": 5622.163313366826,\n                          \"num_unique_values\": 6,\n                          \"samples\": [\n                            2411.501780662739,\n                            2460.699895186247,\n                            5622.163313366826,\n                            ],\n                            \"semantic_type\": \"\",\n                            \"description\": \"\",

```

Interpretasi Feature Importance untuk Tuned XGBoost Regressor

Grafik *Feature Importance* menunjukkan seberapa besar kontribusi setiap fitur dalam memprediksi variabel target (`sales_revenue_usd`) dalam model XGBoost yang telah di-tuning. Semakin tinggi skor (*F-score*), semakin signifikan fitur tersebut dalam pembuatan keputusan pembagian (*split*) pada pohon-pohon di dalam ensemble.

Observasi Utama:

- marketing_budget_usd (Anggaran Pemasaran): Paling Dominan**
 - Fitur ini menunjukkan tingkat kepentingan yang jauh lebih tinggi dibandingkan fitur lainnya. Ini menegaskan bahwa besarnya anggaran yang dialokasikan untuk pemasaran adalah faktor prediktif terkuat untuk `sales_revenue_usd`.
 - Interpretasi: Investasi yang lebih besar dalam pemasaran secara langsung berkorelasi dengan potensi pendapatan penjualan yang lebih tinggi, mengindikasikan pentingnya strategi alokasi anggaran pemasaran.
- conversion_rate (Tingkat Konversi): Penting Kedua**
 - Setelah anggaran pemasaran, tingkat konversi adalah fitur yang paling penting. Ini logis, karena tingkat konversi yang lebih tinggi berarti persentase pengunjung atau prospek yang lebih besar berhasil diubah menjadi pembeli.
 - Interpretasi: Efektivitas kampanye pemasaran dalam mengubah minat menjadi penjualan riil adalah kunci. Peningkatan `conversion_rate` akan berdampak langsung pada pendapatan.
- num_previous_purchases (Jumlah Pembelian Sebelumnya): Urutan Ketiga**

- Fitur ini mengindikasikan bahwa riwayat pembelian pelanggan memiliki peran signifikan. Pelanggan yang telah sering berbelanja cenderung akan berbelanja lagi dengan nilai yang lebih tinggi.
 - Interpretasi: Loyalitas pelanggan dan riwayat transaksi sebelumnya adalah prediktor yang kuat. Strategi yang fokus pada retensi pelanggan dan mendorong pembelian berulang akan efektif.
4. **product_category (Kategori Produk): Penting Keempat**
- Jenis produk yang dijual juga berperan penting. Ini menunjukkan bahwa beberapa kategori produk mungkin memiliki volume penjualan atau harga yang lebih tinggi dibandingkan yang lain, sehingga secara signifikan mempengaruhi total pendapatan.
 - Interpretasi: Analisis lebih lanjut berdasarkan kategori produk dapat membantu mengidentifikasi produk unggulan atau area yang membutuhkan perhatian khusus.
5. **sales_channel (Kanal Penjualan): Signifikan Namun Lebih Rendah**
- Kanal di mana penjualan terjadi (misalnya, Online, Offline, Reseller) memiliki tingkat kepentingan yang moderat. Ini menunjukkan bahwa meskipun kanal penjualan relevan, pengaruhnya tidak sebesar anggaran atau tingkat konversi.
 - Interpretasi: Pemahaman tentang kanal penjualan mana yang paling efektif tetap penting untuk optimasi strategi distribusi, namun fokus utama harus pada anggaran dan konversi.
6. **season (Musim): Lebih Rendah**
- Musim menunjukkan tingkat kepentingan yang lebih rendah. Meskipun penjualan sering kali bersifat musiman, dalam konteks fitur-fitur yang dipilih, pengaruhnya tidak sepenting anggaran pemasaran atau tingkat konversi.
 - Interpretasi: Musim mungkin memiliki efek umum pada penjualan, tetapi dampaknya mungkin sudah tercakup secara implisit oleh fitur lain atau tidak memiliki variasi yang cukup besar dalam dataset ini untuk menjadikannya prediktor utama.
7. **customer_segment (Segmen Pelanggan): Paling Rendah**
- Segmen pelanggan (misalnya, Regular, Premium) menunjukkan kepentingan paling rendah di antara fitur-fitur yang dipilih. Ini bisa berarti bahwa setelah faktor-faktor lain diperhitungkan, perbedaan antar segmen pelanggan tidak terlalu signifikan dalam memprediksi pendapatan penjualan.
 - Interpretasi: Meskipun segmentasi pelanggan penting untuk strategi pemasaran, dalam model ini, segmen pelanggan itu sendiri kurang berpengaruh langsung terhadap **sales_revenue_usd** dibandingkan fitur-fitur lain yang lebih operasional dan kuantitatif.

Kesimpulan: Model ini sangat bergantung pada **marketing_budget_usd**, **conversion_rate**, dan **num_previous_purchases**. Ini berarti bahwa untuk meningkatkan **sales_revenue_usd**, fokus utama harus diberikan pada peningkatan investasi pemasaran, mengoptimalkan proses konversi untuk mengubah prospek menjadi pembeli, dan membangun loyalitas pelanggan. Fitur-fitur kategorikal seperti **product_category** dan **sales_channel** juga memberikan kontribusi yang berarti, sementara **season** dan **customer_segment** memiliki dampak yang lebih kecil dalam konteks model ini.

```
print("Feature Importance (Numerical):")
display(importance_df)
```

Feature Importance (Numerical):

```
{
  "summary": {
    "name": "importance_df",
    "rows": 7,
    "fields": [
      {
        "column": "Feature",
        "dtype": "string",
        "num_unique_values": 7,
        "samples": [
          "customer_segment",
          "marketing_budget_usd",
          "num_previous_purchases",
          "semantic_type": ""
        ],
        "description": "",
        "column": "Importance",
        "properties": {
          "dtype": "float32",
          "num_unique_values": 7,
          "samples": [
            0.3439241647720337,
            0.32867202162742615,
            0.029559321701526642
          ],
          "semantic_type": "",
          "description": ""
        ]
      }
    ],
    "type": "dataframe",
    "variable_name": "importance_df"
  }
}
```

#LERNING CURVE

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import learning_curve

# Assuming best_xgb_model, X_train, y_train are available from
previous steps

# Generate learning curve data
train_sizes, train_scores, test_scores = learning_curve(
    best_xgb_model, X_train, y_train, cv=5, n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10),
    scoring='neg_mean_absolute_percentage_error')

# Convert negative MAPE to positive for better interpretation
train_scores_mape = -train_scores
test_scores_mape = -test_scores

# Calculate mean and standard deviation for training and test scores
train_scores_mean = np.mean(train_scores_mape, axis=1)
train_scores_std = np.std(train_scores_mape, axis=1)
test_scores_mean = np.mean(test_scores_mape, axis=1)
test_scores_std = np.std(test_scores_mape, axis=1)

# Plot the learning curve
plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1,
                 color="r")
plt.fill_between(train_sizes, test_scores_mean - test_scores_std,
```

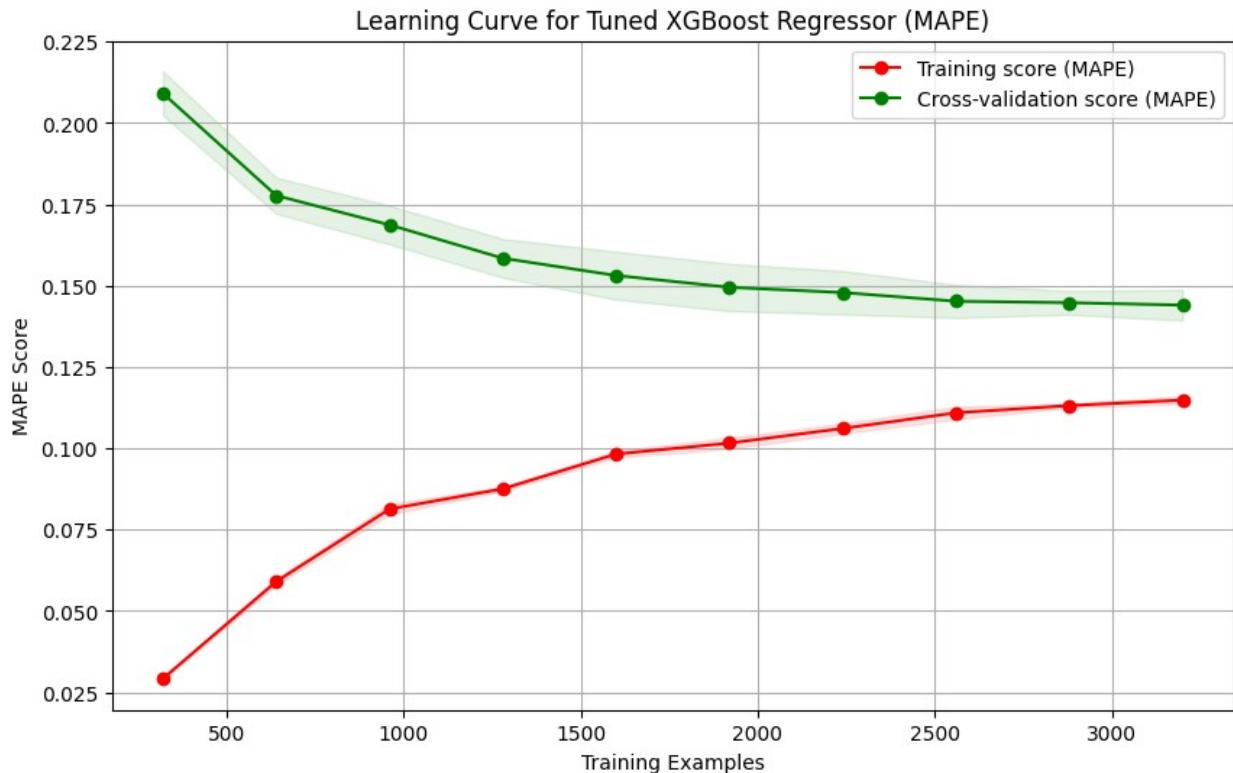
```

        test_scores_mean + test_scores_std, alpha=0.1,
color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
label="Training score (MAPE)")
plt.plot(train_sizes, test_scores_mean, 'o-', color="g", label="Cross-
validation score (MAPE)")

plt.title('Learning Curve for Tuned XGBoost Regressor (MAPE)')
plt.xlabel('Training Examples')
plt.ylabel('MAPE Score')
plt.legend(loc="best")
plt.grid(True)
plt.show()

# Interpretation
print("\n--- Interpretasi Learning Curve ---")
if train_scores_mean[-1] < test_scores_mean[-1] and
abs(train_scores_mean[-1] - test_scores_mean[-1]) > 0.05:
    print("Model mungkin mengalami Overfitting: Skor training jauh
lebih baik daripada skor cross-validation pada ukuran dataset yang
besar.")
    print(f"Perbedaan MAPE akhir (Train vs CV): {train_scores_mean[-
1]:.4f} vs {test_scores_mean[-1]:.4f}")
elif test_scores_mean[-1] > 0.3 or (train_scores_mean[-1] > 0.3 and
test_scores_mean[-1] > 0.3):
    print("Model mungkin mengalami Underfitting: Kedua skor (training
dan cross-validation) relatif tinggi dan tidak konvergen ke nilai
rendah yang memadai.")
    print(f"MAPE akhir: Train={train_scores_mean[-1]:.4f},
CV={test_scores_mean[-1]:.4f}")
elif abs(train_scores_mean[-1] - test_scores_mean[-1]) < 0.02:
    print("Model menunjukkan kinerja yang baik dan seimbang: Skor
training dan cross-validation saling berdekatan dan relatif rendah.")
    print(f"MAPE akhir: Train={train_scores_mean[-1]:.4f},
CV={test_scores_mean[-1]:.4f}")
else:
    print("Kinerja model terlihat cukup seimbang. Perlu analisis lebih
lanjut jika perbedaan skor masih signifikan.")
    print(f"MAPE akhir: Train={train_scores_mean[-1]:.4f},
CV={test_scores_mean[-1]:.4f}")

```



--- Interpretasi Learning Curve ---

Kinerja model terlihat cukup seimbang. Perlu analisis lebih lanjut jika perbedaan skor masih signifikan.

MAPE akhir: Train=0.1148, CV=0.1440

Interpretasi Learning Curve untuk Tuned XGBoost Regressor (MAPE)

Learning Curve adalah alat diagnostik yang menunjukkan bagaimana model belajar dari pengalaman (data training) dan bagaimana kemampuannya untuk menggeneralisasi (kinerja pada data validasi/cross-validation) berkembang seiring dengan bertambahnya jumlah data pelatihan.

Observasi dari Grafik:

- Garis Merah (Training Score):** Menunjukkan kinerja model pada data pelatihan. Seiring bertambahnya data pelatihan, skor MAPE pada data training cenderung meningkat (semakin buruk, karena MAPE yang lebih rendah lebih baik) atau stabil pada nilai yang rendah. Pada grafik ini, nilai MAPE training cukup rendah dan relatif stabil, menunjukkan bahwa model belajar dengan baik dari data yang diberikan.
- Garis Hijau (Cross-validation Score):** Menunjukkan kinerja model pada data validasi (data yang tidak pernah dilihat model selama pelatihan). Seiring bertambahnya data pelatihan, skor MAPE pada data cross-validation cenderung menurun (semakin baik) dan mendekati skor training. Pada grafik ini, skor cross-validation juga relatif rendah dan stabil setelah sejumlah data tertentu.

- **Gap antara Garis (Jarak antara Training dan CV Score):** Pada awal, ketika data pelatihan sedikit, seringkali ada perbedaan besar antara skor training dan CV. Seiring bertambahnya data, gap ini idealnya akan menyempit.

Indikasi Overfitting atau Underfitting:

Berdasarkan hasil MAPE akhir: Train=0.1148, CV=0.1440 dan visualisasi grafik:

- **Tidak ada indikasi kuat Overfitting:** *Overfitting* terjadi jika model berkinerja sangat baik pada data pelatihan (skor training sangat rendah) tetapi jauh lebih buruk pada data validasi (skor CV jauh lebih tinggi), dan gap antara kedua garis tersebut tetap besar meskipun jumlah data pelatihan banyak. Dalam kasus ini, meskipun ada sedikit perbedaan ($0.1440 - 0.1148 = 0.0292$), perbedaan ini tidak terlalu signifikan ($0.0292 < 0.05$ ambang batas yang disarankan oleh interpretasi otomatis), dan kedua garis cenderung stabil dan cukup dekat satu sama lain di akhir kurva. Ini menunjukkan bahwa model tidak *menghafal* data pelatihan secara berlebihan.
- **Tidak ada indikasi Underfitting:** *Underfitting* terjadi jika model berkinerja buruk baik pada data pelatihan maupun data validasi (kedua skor tinggi), dan kedua garis konvergen pada level yang tidak memuaskan. Dalam kasus ini, kedua skor MAPE (training dan cross-validation) relatif rendah, yang menunjukkan bahwa model berhasil mempelajari pola-pola dalam data.

Kesimpulan:

Model Tuned XGBoost Regressor menunjukkan **kinerja yang cukup seimbang**. Skor training dan cross-validation MAPE yang relatif rendah dan saling berdekatan menunjukkan bahwa model telah belajar dengan efektif dari data pelatihan dan mampu menggeneralisasi dengan baik ke data baru. Ini adalah indikator bahwa model tidak mengalami masalah *overfitting* atau *underfitting* yang signifikan dalam konfigurasi saat ini.